NEOLABS

SECURITY LABS · SOC LEVEL 1

SOC Level 1 Complete Toolkit

Analyst handbook, Wazuh guide, templates, references and guided lab

Version: week1-prod-240

Publication date: 2026-10-31

Classification: Student Training Material

Authorised synthetic training use only.

Contents

1. Module 1 — Security Operations, SOC Models and Analyst Roles
2. Module 2 — Alert Triage, Evidence and Escalation
3. Module 3 — SIEM Pipelines, Data Quality and Detection Context
4. Module 4 — Incident Response and Playbook Development
5. Module 5 — Windows and Sysmon Investigation
6. Module 6 — Linux, Web and Cloud Log Investigation
7. Module 7 — Wazuh Alert Investigation and Safe Tuning
8. Module 8 — Case Management, Reporting and SOC Capstone
9. Wazuh SOC Level 1 Handbook — Module 1
10. Wazuh Dashboard Tutorial 01 — Orientation and First Investigation
11. NeoLabs SOC L1 Workstation Compatibility
12. NeoLabs SOC L1 Backup and Recovery
13. NeoLabs Wazuh Setup and Troubleshooting Guide
14. NeoLabs Student Wazuh Stack
15. NeoLabs SOC Evidence Log Template
16. NeoLabs SOC Query Journal Template
17. NeoLabs SOC Incident Report Template
18. Practice Lab 01 — Authentication Failure Chain
19. NeoLabs SOC Level 1 Query and Command Reference

Source: docs/secops-foundations/01-security-operations-foundations.md

Module 1 — Security Operations, SOC Models and Analyst Roles

> **NeoLabs SOC Level 1 principle:** A dashboard can display security information, but a Security Operations Center is the people, processes, evidence, technology and authority used to make and act on security decisions.

Module version: 0.2-draft
Review baseline: 1 August 2026
Expected study time: 3–4 hours

Learning objectives

By the end of this module, a learner should be able to:

- define Security Operations and explain why a SOC is more than a room or SIEM dashboard;
- describe internal, outsourced, hybrid and virtual SOC models;
- distinguish the typical responsibilities of SOC Level 1, Level 2 and Level 3 personnel;
- use the terms log, event, alert, case, finding, incident, evidence, IOC and IOA correctly;
- distinguish severity, priority, confidence and business impact;
- explain the boundary between investigation, escalation and authorised response;
- describe a normal SOC shift from handover to closure.

---

1. What Security Operations means

Professional definition

Security Operations, commonly shortened to **SecOps**, is the continuing organisational capability used to monitor technology environments, identify suspicious or harmful activity, investigate security-relevant events, coordinate response and improve defensive controls.

SecOps is continuous because organisational systems continue generating activity outside normal office hours. Even a small organisation that does not operate a 24-hour staffed SOC still needs processes for receiving alerts, deciding urgency and contacting someone who is authorised to act.

Plain-language explanation

SecOps is the part of cybersecurity that asks:

1. What is happening in our systems right now?
2. Is any of it unsafe, unauthorised or unusual enough to investigate?
3. What evidence supports that conclusion?
4. Who needs to know or act?
5. What should change so the same problem is detected or prevented more effectively next time?

A SOC supports those questions with analysts, procedures, logging, SIEM or XDR platforms, endpoint tools, network data, cloud audit trails, ticketing systems, communication channels and management authority.

People, process and technology

Element	What it contains	Why it matters
People	Analysts, incident responders, threat hunters, engineers, IT staff, legal, management and system owners	Tools cannot decide business impact or authorise every response action
Process	Triage steps, severity definitions, escalation paths, playbooks, evidence handling and reporting	A repeatable process reduces inconsistent decisions during pressure
Technology	SIEM, Wazuh, endpoint protection, firewalls, cloud logs, threat intelligence and case systems	Technology collects and organises evidence at a scale people cannot review manually

A mature SOC keeps these elements aligned. Buying a SIEM without staffing, useful logs or escalation procedures creates an expensive dashboard rather than an effective security operation.

---

2. What a Security Operations Center is

A **Security Operations Center (SOC)** is the team and operating structure that centralises security monitoring and incident-handling activities. “Center” does not require a physical room. A distributed team can operate a virtual SOC if it has defined responsibilities, secure tools, reliable communication and an established chain of authority.

Core SOC functions

A SOC commonly performs some combination of the following:

- monitoring alert queues and important data sources;
- validating that required telemetry is arriving;

- triaging and classifying alerts;
- investigating activity across identities, hosts, applications, networks and cloud resources;
- documenting evidence and maintaining case records;
- escalating confirmed or unresolved risk;
- coordinating containment and recovery with authorised teams;
- tuning detections and reducing avoidable noise;
- measuring coverage, workload and response performance;
- preserving lessons from incidents and near misses.

SOC is not the same as IT support

IT support and SecOps often cooperate, but their primary questions differ.

IT support question	SecOps question
Why can the user not sign in?	Is the failed sign-in activity normal, accidental or hostile?
Why is the server slow?	Is the slowdown associated with scanning, resource abuse or unauthorised execution?
How do we restore the service?	What evidence must be preserved before restoration, and could recovery reintroduce the threat?
Which update fixes the error?	Does the missing update expose a vulnerability that changes incident priority?

One event can require both teams. A locked account might be a normal support issue, a password attack or both.

---

3. Common SOC operating models

3.1 Internal SOC

An **internal SOC** is operated by the organisation’s own employees. It usually has deeper knowledge of the organisation’s systems, business processes and approved administrative behaviour.

Advantages

- strong access to internal context and system owners;
- direct control over tooling and procedures;
- easier alignment with organisational priorities.

Challenges

- staffing a 24-hour operation can be expensive;
- specialist expertise may be difficult to recruit;
- small teams can experience alert fatigue and coverage gaps.

3.2 Managed Security Service Provider or Managed SOC

A **Managed Security Service Provider (MSSP)** or managed SOC monitors systems for multiple customer organisations under a service agreement.

Advantages

- broader staffing coverage;
- access to established platforms and specialist personnel;
- useful for organisations that cannot build a complete internal SOC.

Challenges

- analysts may initially lack business context;
- customer and provider responsibilities must be clearly divided;
- escalation and evidence-sharing delays can affect response.

3.3 Hybrid SOC

A **hybrid SOC** combines internal capability with an external provider. For example, a provider may monitor overnight alerts while internal staff handle daytime investigations and response decisions.

This model works only when ownership is explicit. An alert should never remain unresolved because both parties assumed the other was responsible.

3.4 Virtual or distributed SOC

A **virtual SOC** operates across different locations without requiring one physical room. It still needs controlled access, shift handover, case tracking, secure communications and an on-call structure.

3.5 Follow-the-sun model

Large organisations may hand monitoring between teams in different time zones. This provides continuous coverage but makes clear case notes and handover quality essential.

---

4. SOC roles and tier boundaries

Role names differ between organisations. “Level 1” does not always describe exactly the same permissions or responsibilities. The VCC programme uses the following learning model.

4.1 SOC Level 1 — monitoring and triage

A Level 1 analyst is commonly the first human reviewer of an alert. The analyst’s job is not to prove every incident alone. It is to perform a reliable first investigation and make the next decision defensible.

Typical responsibilities include:

- review the alert and underlying raw event;
- verify the source, time and affected entity;
- determine whether expected telemetry is present;
- collect basic context about the account, host, application or resource;
- search a reasonable time window for related activity;
- distinguish obvious benign causes from unresolved or suspicious activity;
- classify the alert using the organisation’s definitions;
- document facts, interpretation, uncertainty and queries performed;
- escalate when scope, impact, permissions or complexity exceed Level 1 authority.

In the VCC programme, interns may recommend an action but do not perform unauthorised containment against lab infrastructure.

4.2 SOC Level 2 — deeper investigation and coordinated response

Level 2 analysts usually handle escalated cases requiring broader correlation or response coordination. Activities may include:

- reconstructing a detailed timeline across several data sources;
- scoping affected identities and assets;
- validating whether an attack succeeded;
- coordinating approved containment with IT, cloud or application teams;
- collecting additional forensic evidence;
- determining root cause and likely entry path;
- updating stakeholders and case severity;
- identifying detection gaps.

4.3 SOC Level 3 — threat hunting, advanced analysis and detection engineering

Level 3 titles can include threat hunter, senior analyst, malware analyst or detection engineer. Typical work may include:

- hypothesis-driven hunting beyond existing alerts;

- advanced endpoint, network or malware analysis;
- researching adversary behaviour;
- creating and validating new detections;
- leading complex incidents;
- improving playbooks and architecture;
- mentoring junior analysts.

4.4 Supporting roles

A SOC depends on people outside the tier structure:

- **SOC manager:** staffing, process ownership, metrics and escalation authority;
- **SIEM/platform engineer:** ingestion pipelines, parsers, storage, upgrades and platform health;
- **detection engineer:** threat hypotheses, analytics, validation and tuning;
- **incident commander:** coordinates a major incident and keeps decision-making organised;
- **forensic examiner:** acquires and analyses evidence using controlled methods;
- **threat intelligence analyst:** provides adversary, campaign and indicator context;
- **system or application owner:** explains expected behaviour and business impact;
- **legal/privacy/communications:** advises on obligations and external communication.

---

5. Core terminology

5.1 Log

A **log** is a stored record produced by a system, application, service, device or security tool.

Example:

```
2026-08-01T09:14:21Z web01 sshd[24109]: Failed password for invalid user admin from 203.0.113.44 port 44218 ssh2
```

The record states that an SSH authentication attempt failed. It does not identify the human behind the source address and does not prove compromise.

5.2 Event

An **event** is a specific occurrence represented by one or more log records. A successful sign-in, file change, process start or cloud API request can be an event.

5.3 Alert

An **alert** is a notification produced when a rule, analytic, threshold or model decides that an event or sequence deserves analyst attention.

An alert is a lead, not a verdict. A rule can match correctly while the underlying activity remains authorised.

5.4 Case or ticket

A **case** or **ticket** is the organised record of analyst work. It normally stores the alert, affected entities, evidence, timeline, queries, decisions, communications and next actions.

5.5 Finding

A **finding** is a statement supported by analysed evidence.

Weak finding:

> The user was hacked.

Better finding:

> Authentication logs recorded 47 failed sign-ins for `learner-17` from `203.0.113.44`, followed by one successful sign-in from the same source. The account's normal source range is not available, so unauthorised access is suspected but not yet confirmed. Confidence: medium.

5.6 Incident

A **cybersecurity incident** is an occurrence that actually or imminently jeopardises information or systems, violates security policy, or requires coordinated response according to the organisation's definition.

Not every alert becomes an incident. Organisations should document their own declaration criteria.

5.7 Evidence

Evidence is information preserved and interpreted to support or challenge a conclusion. Good evidence records include source, time, collection method, relevant fields and integrity considerations.

A screenshot can support a report, but the underlying event or export is usually stronger because it is searchable and contains more fields.

5.8 Indicator of Compromise and Indicator of Attack

An **Indicator of Compromise (IOC)** is an observable value associated with known or suspected malicious activity, such as a file hash, domain, IP address or registry path.

An **Indicator of Attack (IOA)** focuses on behaviour that may show an attack in progress, such as repeated credential guessing followed by unusual access.

IOC caution: addresses, domains and hashes can be shared, reassigned, spoofed or used in benign contexts. An IOC match requires validation.

---

6. Severity, priority, impact and confidence

These terms are related but not interchangeable.

Severity

Severity describes the potential or confirmed seriousness of the security condition. An organisation may calculate severity from technical impact, affected asset criticality, data sensitivity and scope.

Priority

Priority determines the order and urgency of work. A technically severe event may receive lower immediate priority if it is fully contained, while a medium-severity alert affecting an executive account may require immediate review.

Business impact

Business impact describes consequences to operations, finances, safety, legal obligations, reputation or customers.

Confidence

Confidence describes how strongly available evidence supports the current conclusion.

Confidence	Appropriate use
Low	Limited evidence, major visibility gaps or several plausible explanations
Medium	Multiple supporting observations but missing confirmation or context
High	Strong corroborated evidence with few reasonable alternatives

Confidence is not the same as severity. A potentially critical incident can begin with low-confidence evidence and still require urgent escalation.

Worked example

An alert reports a new administrator account on a test server.

- **Potential severity:** High, because unauthorised administrator creation can enable control of the host.
- **Current confidence:** Medium, because the creation event is confirmed but change approval has not been checked.
- **Priority:** High until the system owner confirms whether the account was expected.
- **Impact:** Unknown; no activity by the new account has yet been identified.

---

7. A normal SOC shift

7.1 Handover

The incoming analyst reviews:

- open high-priority cases;
- pending owner responses;
- known maintenance windows;
- platform or data-source outages;
- threat advisories relevant to monitored systems;
- actions that require follow-up.

A good handover states what is known, what remains uncertain and the next action. “Still investigating” is not enough.

7.2 Platform and queue health

Before trusting the alert queue, the analyst checks that the monitoring system is healthy:

- are agents or feeds connected?
- are event volumes within an expected range?
- are timestamps current?
- are parsing fields populated?
- are index or storage errors present?

No alerts can mean no threats, a quiet environment or a broken telemetry pipeline.

7.3 Alert triage

The analyst opens an alert, reviews raw evidence, enriches context, searches related events and decides whether to close, continue or escalate.

7.4 Case documentation

Work is recorded during the investigation, not reconstructed from memory at the end. Queries, time ranges and important negative results should be documented.

7.5 Escalation and communication

The analyst follows the defined escalation route, supplies relevant evidence and avoids overstating conclusions.

7.6 Shift closure

Before handover, the analyst updates open cases, identifies deadlines and records platform issues. Unwritten knowledge is easily lost between shifts.

---

8. Worked scenario — failed sign-ins followed by success

Alert summary

```
Detection: Repeated authentication failures followed by success
Account: svc-backup
Destination: app-pod-03
First failure: 2026-08-01T09:14:21Z
Success: 2026-08-01T09:18:07Z
Source: 203.0.113.44
```

Initial questions

1. Is `svc-backup` expected to sign in interactively?
2. Is the source address expected for the service?
3. What authentication method was used?
4. What happened after the success?
5. Did the same source target other accounts?
6. Was there a maintenance or deployment window?
7. Are the timestamps and source fields trustworthy?

Observation, interpretation and conclusion

Observation: The monitored authentication source recorded 19 failures for `svc-backup` from `203.0.113.44`, followed by one successful authentication from the same source.

Interpretation: A failure-to-success sequence can occur during password guessing, but it can also result from a stored credential being corrected or an automation retrying after a secret rotation.

Current conclusion: Suspicious authentication requiring escalation unless approved maintenance explains the sequence. Confidence: medium.

Next evidence: Post-authentication activity, deployment records, service owner confirmation and other accounts targeted by the source.

---

9. Common beginner mistakes

Mistake 1 — treating the alert title as evidence

The title is a summary produced by detection logic. Always inspect the records that caused it.

Mistake 2 — assuming unusual means malicious

An anomaly differs from the baseline. It becomes suspicious or malicious only when context and evidence support that judgment.

Mistake 3 — using “false positive” for every benign alert

A rule may correctly detect behaviour that is real but authorised. Some teams call this a **benign positive** rather than a false positive. Use the organisation’s definitions consistently.

Mistake 4 — escalating without a useful handoff

“Please investigate” transfers work but not understanding. Include the trigger, affected entities, timeline, searches, evidence, classification and unresolved questions.

Mistake 5 — taking response actions outside authority

Disabling accounts, blocking addresses or isolating systems can affect operations and evidence. Follow the authorised process.

Mistake 6 — ignoring telemetry health

Missing data is not evidence that nothing happened.

---

10. Guided exercise

For each statement, identify whether it is an observation, interpretation or conclusion.

1. `Event ID 4625 recorded a failed network logon for account student03 from 203.0.113.80.`
2. `The source address is outside the account's documented normal range.`
3. `The activity is suspicious and should be escalated for possible password guessing.`
4. `No process-creation events were found in the available five-minute window.`
5. `Endpoint process auditing may be disabled, so the absence of process events does not rule out execution.`

Suggested discussion:

- Statements 1 and 4 are observations.
- Statements 2 and 5 are interpretations based on context or visibility.
- Statement 3 is a conclusion and action decision.

---

11. Review questions

1. Why is a SOC more than a SIEM dashboard?
2. What is the practical difference between an event and an alert?
3. What should a Level 1 analyst normally include in an escalation?
4. Why can severity be high while confidence remains low?
5. Give one example of an IOC and one limitation of IOC-based decisions.
6. What does a quiet alert queue fail to prove?
7. Explain the difference between observation, interpretation and conclusion.
8. When should a Level 1 analyst stop investigating and escalate?

References

- NIST SP 800-61 Rev. 3, *Incident Response Recommendations and Considerations for Cybersecurity Risk Management*.
- NIST SP 800-92, *Guide to Computer Security Log Management*.
- NIST SP 800-92 Rev. 1 Initial Public Draft, *Cybersecurity Log Management Planning Guide*.
- MITRE ATT&CK v19.1.
- Wazuh official documentation, architecture and data-analysis sections.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.

Source: docs/secops-foundations/02-alert-triage-evidence-and-escalation.md

Module 2 — Alert Triage, Evidence and Escalation

> **NeoLabs SOC Level 1 principle:** Triage is a disciplined decision process. It is not a race to close alerts and it is not a requirement to solve every incident alone.

Module version: 0.2-draft
Review baseline: 1 August 2026
Expected study time: 4–5 hours

Learning objectives

By the end of this module, a learner should be able to:

- explain the purpose and limits of SOC Level 1 triage;
- apply a repeatable alert-review workflow;
- verify the evidence behind an alert before accepting its title;
- identify affected entities, scope, time window and data-source health;
- distinguish true positive, benign positive, false positive and insufficient evidence;
- record queries, negative results, assumptions and confidence;
- prepare a useful escalation package;
- recognise stop conditions that require immediate escalation.

---

1. What triage means

In medicine, triage determines who needs attention first. In a SOC, **alert triage** is the initial process used to decide what an alert represents, how urgent it is, whether more investigation is needed and who should handle the next action.

Triage answers four practical questions:

1. **Is the alert based on valid and trustworthy data?**
2. **What entity and activity are involved?**
3. **How concerning is the activity in context?**
4. **Should the alert be closed, monitored, investigated further or escalated?**

A Level 1 analyst should reach the best supported decision possible within the allotted time and authority. Spending hours on a case that clearly requires a higher tier can delay response.

2. The NeoLabs L1 triage workflow

Use the following sequence unless a playbook requires a different order.

```
Receive alert
  → verify source and raw event
  → establish time, identity, asset and action
  → validate telemetry health
  → gather immediate context
  → search related activity
  → consider benign explanations
  → classify and assign confidence
  → document and close or escalate
```

Step 1 — Read the alert without trusting the conclusion

Record:

- alert name and identifier;
- creation time;
- detection or rule identifier;
- severity assigned by the tool;
- data source;
- affected user, host, application, IP, file or cloud resource;
- the raw event or events that triggered it.

The alert severity is input to the investigation, not the final decision.

Step 2 — Verify the raw evidence

Ask:

- does the raw record exist?
- did the expected parser or decoder extract the fields correctly?
- is the timestamp event time or ingest time?
- is the source field the original client, a proxy or a collector?
- is the alert based on one record, a threshold or a sequence?
- could duplicate ingestion have inflated the count?

Example:

An alert says “20 failed SSH logins.” Inspection reveals the same event was ingested twice by two collectors, meaning there were 10 attempts, not 20. The activity may still be suspicious, but the analyst must correct the scope.

Step 3 — Identify the entities

An **entity** is something involved in the activity and useful for correlation.

Common entities include:

- user account or service account;
- source and destination IP address;
- hostname, device ID, agent ID or cloud resource;
- process image, PID, Process GUID or command line;
- file path and hash;
- URL, domain, request ID or session ID;
- role, API key identifier or cloud account;
- pod identifier in the VCC lab.

Record stable identifiers where available. A display name may change; a SID, ARN, agent ID or request ID can be more reliable.

Step 4 — Establish the investigation window

Start with a narrow window around the alert, then expand only when evidence requires it.

Example approach:

- 5 minutes before and after for a single process or web request;
- 15–30 minutes for an authentication sequence;
- several hours for account takeover or cloud configuration changes;
- days for low-frequency persistence or slow credential abuse.

Do not assume records are perfectly ordered. Clock drift, delayed forwarding and different time zones can move events outside the expected sequence.

Step 5 — Validate telemetry health

Before interpreting missing evidence, determine whether the source could have produced and delivered it.

Check:

- agent or feed connection state;
- last event time;
- normal event volume;
- audit policy or log configuration;
- parser/decoder success;
- index availability and retention;
- system clock and time zone.

Important distinction:

- **No evidence found** means the search did not return the event.
- **Evidence that no activity occurred** requires a source that reliably records the activity and was functioning during the relevant period.

Step 6 — Gather context

Useful context can include:

- asset criticality and owner;
- account role and normal function;
- normal source locations or peer group;
- maintenance and deployment windows;
- approved vulnerability scanning;
- known automation or scheduled tasks;
- threat-intelligence matches;
- recent password resets or configuration changes;
- prior alerts involving the same entity.

Context should explain behaviour, not automatically excuse it. “It is an administrator account” can increase risk rather than make unusual activity harmless.

Step 7 — Search for related activity

Good pivots include:

- same user across other hosts;
- same source IP against other users;
- same host before and after the alert;
- process parent and child relationships;
- file hash or path across endpoints;
- request ID across proxy, application and database logs;
- cloud access-key or role session across API events;
- same pod and correlation ID across VCC telemetry.

Record both the query and time range. “Checked logs” is not reproducible.

Step 8 — Consider alternative explanations

A strong analyst actively tests benign and malicious explanations.

For repeated login failures, alternatives may include:

- password guessing;
- a user typing an old password;
- a service using an expired secret;
- a vulnerability scanner;
- a disconnected system repeatedly retrying;
- an attacker testing leaked credentials.

The goal is not to invent excuses. It is to avoid closing on the first plausible story.

Step 9 — Classify and assign confidence

Use the programme’s approved categories. A practical training model is:

Classification	Meaning
False positive	Detection logic or parsing produced an incorrect match
Benign positive	Detection correctly identified the behaviour, but the activity was authorised or expected
Suspicious	Evidence justifies concern, but malicious activity is not confirmed
Confirmed incident	Evidence supports unauthorised or harmful activity requiring coordinated response
Insufficient evidence	Available telemetry or context cannot support a reliable classification
Data-quality issue	The primary problem is missing, duplicated, delayed or incorrectly parsed telemetry

Add a confidence statement and explain why.

Step 10 — Document and act

A case record should allow another analyst to continue without repeating all work.

At minimum include:

- alert and affected entities;
- event and ingest times;
- observed facts;
- queries and time ranges;
- correlated evidence;
- relevant negative findings;
- alternative explanations considered;
- current classification and confidence;
- recommended next action;
- reason for closure or escalation.

3. What evidence proves, suggests and cannot prove

Example 1 — Firewall allow record

```
{
  "event_time": "2026-08-01T10:02:44Z",
  "action": "ACCEPT",
  "src_ip": "203.0.113.9",
  "src_port": 53120,
  "dst_ip": "10.20.3.15",
  "dst_port": 443,
  "protocol": "TCP"
}
```

Proves: The firewall recorded an allowed flow with those observed network fields.

Suggests: A connection may have been established between the source and destination.

Does not prove: Which human initiated it, what application data was exchanged, whether authentication succeeded or whether exploitation occurred.

Example 2 — HTTP 200 response

```
203.0.113.44 - - [01/Aug/2026:10:06:22 +0000] "GET /api/profile/204 HTTP/1.1" 200 894
```

Proves: The web server returned HTTP status 200 for the logged request.

Suggests: The route processed successfully from the server’s perspective.

Does not prove: The requester was authorised to access profile 204, that the response contained sensitive data or that the client received the full response.

Example 3 — Antivirus detection

An antivirus alert proves that the product identified content or behaviour matching its detection logic. It does not automatically prove successful execution, persistence or impact. Quarantine status, process telemetry, file origin and surrounding activity are still required.

---

4. Alert dispositions in detail

False positive

Use this when the detection itself is wrong or misleading.

Examples:

- a parser placed the destination IP in the source field;
- a rule matched a harmless string because it lacked boundaries;
- duplicate records caused a threshold to trigger incorrectly;
- a test record was accidentally classified as production activity.

Document the detection problem so it can be tuned.

Benign positive

The rule correctly detected the behaviour, but context shows it was authorised.

Example:

A rule detects a new scheduled task. Change records confirm an approved backup agent installed it, the binary is signed, the path is expected and the timing matches the deployment window.

The event is not a false positive because the scheduled task genuinely existed.

Suspicious activity

The evidence is concerning but incomplete.

Example:

A service account logged in interactively from a new source and launched a shell, but account ownership and maintenance context have not been confirmed.

Confirmed incident

The available evidence satisfies the organisation’s incident criteria.

Example:

An unauthorised account successfully accessed a protected resource, changed configuration and attempted to remove audit records. Multiple sources corroborate the sequence.

Insufficient evidence

Use this when visibility gaps prevent a defensible result. Do not disguise uncertainty as a benign closure.

Example:

A suspected process-execution alert cannot be validated because process auditing was disabled and the endpoint was rebuilt before evidence collection.

Data-quality issue

Sometimes the incident is the monitoring failure itself.

Examples:

- a critical application stopped sending authentication logs;
- event times are shifted by two hours;
- a new schema caused decoder fields to become empty;
- records are being ingested twice.

---

5. Escalation

5.1 Why escalation exists

Escalation transfers a case to someone with additional authority, expertise, tooling or time. It should reduce delay, not merely move responsibility.

5.2 Common escalation triggers

Escalate when:

- activity may affect a critical asset or privileged identity;
- a successful compromise is possible or confirmed;
- scope appears larger than one user or host;
- containment may be required;
- malware, persistence or lateral movement is suspected;
- sensitive or regulated data may be involved;
- evidence collection requires tools or permissions the analyst lacks;
- the alert matches a mandatory playbook trigger;
- the investigation exceeds the approved Level 1 time limit;
- telemetry gaps prevent a safe closure;
- the analyst is uncertain whether an action is authorised.

5.3 Immediate stop conditions

In the VCC environment, stop and escalate immediately if:

- a task appears to require access outside the assigned student-facing interfaces;
- credentials, private keys or production-looking data appear in evidence;
- telemetry appears to come from another pod;
- a student can change configuration and view another pod’s events;
- a requested action could disrupt the lab host or other students;
- the scenario no longer matches the written scope.

5.4 Escalation package

A useful escalation contains:

```
Case title:
Alert ID and detection:
Current severity / priority:
Affected entities:
Earliest and latest relevant event time:
Observed facts:
Queries and sources reviewed:
Timeline summary:
Current classification:
Confidence and reason:
Alternative explanations considered:
Business or lab impact:
Actions already taken:
Recommended next action:
Unresolved questions:
```

Weak escalation

> Suspicious login. Please check.

Stronger escalation

> Wazuh alert `AUTH-SEQ-0142` identified 19 failed logins and one success for `svc-backup` on `pod-03-app` from `203.0.113.44` between 09:14:21Z and 09:18:07Z. The account is labelled as a service identity, and interactive use is not documented in the available profile. I searched the same source across all assigned pod authentication records for ±30 minutes and found failures against two additional accounts. No approved maintenance note is attached. Post-login process telemetry is unavailable because that source stopped reporting at 09:16Z. Classification: suspicious; confidence medium. Escalation requested for account-owner validation and telemetry-gap review.

---

6. Query and evidence journal

Every meaningful search should be reproducible.

Time run	Data source	Query or filter	Time range	Result summary	Next pivot
10:20Z	Wazuh alerts	`data.user: "svc-backup"`	09:00–10:00Z	20 authentication records	Search same source IP
10:23Z	Wazuh alerts	`data.srcip: "203.0.113.44"`	08:45–10:15Z	Three accounts targeted	Check success and post-login events
10:27Z	Endpoint events	host `pod-03-app`	09:15–09:30Z	No records after 09:16Z	Validate source health

A negative result matters only when the data source and query were capable of finding the activity.

---

7. Worked investigation — possible web-account takeover

Alert

```
{
  "rule": "Successful login after repeated failures",
  "user": "student17",
  "src_ip": "198.51.100.73",
  "destination": "vcc-pod-02",
  "failure_count": 12,
  "success_time": "2026-08-01T11:42:08Z",
  "risk": "high"
}
```

Step 1 — Validate

The analyst confirms 12 distinct failed events and one success. No duplicates are present. The source IP field came from the application after trusted-proxy processing.

Step 2 — Context

The user normally signs in from a different example source range. No scheduled test or password-reset activity is documented.

Step 3 — Correlation

The analyst searches the session ID created by the success and finds:

- profile viewed;
- email-change page opened;
- password reset requested;
- no confirmed profile change;

- session revoked five minutes later by the scenario controller.

Step 4 — Interpretation

The sequence is consistent with attempted account takeover, but the lab record does not identify the actor or prove credentials were obtained through any specific method.

Step 5 — Disposition

Classification: Confirmed synthetic security incident within the assigned VCC scenario.
Confidence: High.
Reason: Successful access from the failure source followed by sensitive account actions, corroborated by session and application records.
Escalation: Provide timeline and affected session to mentor; recommend credential reset and review of other accounts targeted by the source within the scenario.

---

8. Time management in L1 triage

A fixed time limit is organisation-specific. The purpose is to prevent one difficult case from blocking the queue.

A practical approach:

- first 2 minutes: verify alert, source and affected entity;
- next 5–10 minutes: gather immediate context and perform core pivots;
- decision point: close with evidence, continue under a playbook or escalate;
- document throughout.

Do not use a time limit to force a benign answer. If the result remains uncertain, state the uncertainty and escalate appropriately.

---

9. Common mistakes

Searching only alerts

Relevant events may exist in raw or archive data without triggering a rule.

Ignoring time zones

Comparing local application time with UTC cloud time can create a false sequence.

Copying raw logs without analysis

Evidence must be connected to the question. A large dump is not a clear escalation.

Closing because an IOC is not on a threat list

Absence from a list does not make an address or file safe.

Overstating identity

An account name or source IP does not automatically identify the human responsible.

Failing to record negative results

A later analyst needs to know what was searched, where and for what time range.

Recommending destructive action too early

Containment should be proportionate and authorised. Preserve evidence and consider business impact.

---

10. Guided practice

Use the synthetic records in `sample-logs/authentication/` when available.

Prepare:

1. a one-paragraph alert summary;
2. a table of affected entities;
3. three queries and their results;
4. an observation–interpretation–conclusion statement;
5. a classification and confidence level;
6. an escalation package or closure justification.

Do not assume the alert is malicious. Test at least two alternative explanations.

---

11. Review questions

1. Why must an analyst inspect the raw event behind an alert?
2. What is the difference between a false positive and a benign positive?
3. When can a negative search result be treated as meaningful evidence?
4. Name five useful correlation entities.
5. Why should the analyst record time ranges with queries?
6. Give three examples of immediate VCC stop conditions.

7. What information makes an escalation useful to Level 2?
8. Why can a telemetry outage require escalation even when no compromise is confirmed?
9. Classify this statement: “The host stopped reporting after the successful login, so the attacker disabled logging.” What is wrong with it?
10. How should an analyst communicate uncertainty?

References

- NIST SP 800-61 Rev. 3.
- NIST SP 800-92 and SP 800-92 Rev. 1 Initial Public Draft.
- Wazuh official documentation: alert management, event logging, WQL and dashboard navigation.
- MITRE ATT&CK v19.1 Detection Strategies, Analytics and Data Components.
- NeoLabs Log Literacy Manual.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.

Source: docs/secops-foundations/03-siem-pipelines-and-log-quality.md

Module 3 — SIEM Pipelines, Data Quality and Detection Context

> **NeoLabs SOC Level 1 principle:** Before trusting a dashboard result, understand how the source activity became a searchable field and which failures could have altered or removed the evidence.

Module version: 0.2-draft
Review baseline: 1 August 2026
Expected study time: 4–6 hours

Learning objectives

By the end of this module, a learner should be able to:

- define SIEM and explain its relationship to log management, detection and case handling;
- describe generation, collection, transport, parsing, normalisation, enrichment, indexing, detection and alerting;
- distinguish raw events, decoded fields, indexed documents and alerts;
- identify common data-quality failures and their investigative effect;
- explain why event time and ingest time can differ;
- recognise schema drift, duplicate ingestion, clock skew and retention gaps;
- validate a telemetry pipeline before concluding that an event did not occur;
- explain the Wazuh manager, indexer and dashboard roles at a beginner-to-intermediate level.

---

1. What a SIEM is

A **Security Information and Event Management (SIEM)** platform centralises security-relevant data, makes it searchable and applies rules or analytics to identify activity that deserves investigation.

A SIEM normally supports four broad capabilities:

1. **Collection:** receive records from endpoints, applications, network devices, identity systems and cloud services.
2. **Organisation:** parse, normalise, enrich, index and retain records.
3. **Detection:** evaluate records or sequences against rules, thresholds and analytics.

4. **Investigation:** provide searches, visualisations, alert queues and links to evidence.

A SIEM does not create visibility that the source never generated. If process auditing is disabled, the SIEM cannot reconstruct every process from nothing. If an application logs only “request failed” without user, route or request ID, later investigation will be limited.

SIEM, XDR and log management

- **Log management** covers the generation, transmission, storage, access and disposal of log data.
- **SIEM** adds security-oriented correlation, detection, investigation and alerting.
- **XDR** generally combines detection and response across several security domains, such as endpoints, identities, cloud and email. Product definitions differ.

For SOC Level 1 work, the important skill is not memorising product categories. It is understanding which source produced the evidence, how it was transformed and what response authority exists.

---

2. The telemetry pipeline

```
System activity
  → log generation
  → collection
  → transport
  → parsing / decoding
  → normalisation
  → enrichment
  → indexing and retention
  → rule or analytic
  → alert
  → investigation and case record
```

Each stage can succeed, fail or change the meaning available to an analyst.

2.1 System activity

Something occurs: a user signs in, a process starts, a file changes, a web request reaches an API or a cloud role performs an action.

The activity itself is not the log. The log is a system’s recorded representation of that activity.

2.2 Log generation

The source must be configured to record the event.

Examples:

- Windows Security auditing records selected authentication and account events.
- Sysmon records configured process, network, DNS, file and registry activity.
- NGINX records requests according to its log format.
- AWS CloudTrail records supported API activity according to trail and event-selector configuration.
- An application records business events only if developers implemented suitable logging.

Generation failure: Auditing is disabled, the event category is excluded or the application never logs the required action.

2.3 Collection

A collector reads the event from the source.

Collection mechanisms include:

- Wazuh agents;
- Windows Event Forwarding;
- syslog;
- API polling;
- cloud object storage;
- file readers;
- container stdout collectors;
- message queues.

Collection failure: The agent is stopped, lacks permission, watches the wrong path or starts after the event occurred.

2.4 Transport

Records move from the source or collector to the analysis platform.

Security considerations include:

- authentication of the sender and receiver;
- encryption in transit;
- buffering when the destination is unavailable;
- duplicate delivery after retries;
- network and proxy paths;
- size or rate limits.

Transport failure: Network interruption, expired certificate, rejected credentials, queue overflow or dropped oversized records.

2.5 Parsing or decoding

Parsing converts raw text or structured records into fields.

Raw example:

```
Aug 01 09:14:21 web01 sshd[24109]: Failed password for invalid user admin from 203.0.113.44 port 44218
ssh2
```

Possible decoded fields:

```
{
  "host": "web01",
  "program": "sshd",
  "process_id": 24109,
  "event_action": "authentication_failure",
  "user": "admin",
  "user_status": "invalid",
  "source_ip": "203.0.113.44",
  "source_port": 44218,
  "protocol": "ssh2"
}
```

Parsing makes searches more reliable, but the raw record should remain available where policy permits.

Parsing failure: The format changes, the decoder matches the wrong pattern or a field contains unexpected characters.

2.6 Normalisation

Normalisation maps similar meanings from different sources to consistent field names and values.

Source-specific field	Normalised idea
`src`, `client_ip`, `sourceIPAddress`	source IP address
`username`, `TargetUserName`, `userIdentity.arn`	acting or targeted identity, with source-specific detail retained
`result`, `status`, HTTP status	outcome, interpreted according to source semantics

Normalisation helps cross-source searches. It must not erase important differences. An AWS role ARN is not identical to a Windows username even if both represent identity.

2.7 Enrichment

Enrichment adds context not present in the original event.

Examples:

- asset owner and criticality;
- approved pod identifier;
- IP reputation or geographic context;
- vulnerability exposure;
- identity role;
- threat-intelligence tags;
- change-window information.

Enrichment can become stale. A system may have changed owner, a cloud address may be reassigned and a threat-intelligence match may be outdated.

2.8 Indexing and retention

An index stores fields in a form designed for search. Field mappings determine whether values behave as exact keywords, analysed text, numbers, dates or IP addresses.

Examples of mapping problems:

- a numeric status stored as text cannot be reliably compared with numeric ranges;
- an IP stored as ordinary text loses IP-aware queries;
- a timestamp stored in an unrecognised format may sort incorrectly;
- a username analysed into tokens may behave differently from an exact keyword field.

Retention determines how long evidence remains searchable or archived. Retention must consider operational need, incident-discovery delay, privacy, regulation, storage cost and evidence requirements.

2.9 Detection

A detection evaluates records or sequences.

Common detection types:

- exact or pattern match;
- threshold within a time window;
- sequence, such as failures followed by success;
- correlation across sources;
- baseline or anomaly comparison;
- threat-intelligence match;
- model or risk score.

A detection should state its data requirements. A rule that depends on command lines cannot work correctly when command-line auditing is disabled.

2.10 Alerting

The platform creates an alert with fields such as rule, severity, entities, evidence links and timestamps.

The alert may summarise several events. Analysts should determine whether the displayed count represents distinct source events, grouped documents or duplicated ingestion.

---

3. Wazuh components in this toolkit

Wazuh manager

The **Wazuh manager** receives and analyses security data. It runs decoders and rules, manages agent-related functions and produces alert records.

In the NeoLabs student deployment, the manager also monitors a local NDJSON file written by the authorised VCC telemetry collector. The collector obtains only the feed issued to that learner’s assigned pod.

Wazuh indexer

The **Wazuh indexer** stores and indexes alert and security data for search and visualisation. It is based on OpenSearch technology.

The indexer should not be exposed publicly in the student profile. The dashboard and internal components access it over the private Compose network.

Wazuh dashboard

The **Wazuh dashboard** provides the analyst interface for alerts, agents, security modules, searches and visualisations.

A dashboard view is not the evidence itself. The analyst should be able to open event details and identify the supporting fields.

Wazuh agent

The **Wazuh agent** is installed on monitored endpoints to collect local telemetry and perform supported security functions.

VCC interns do not install agents on pod infrastructure. Pod telemetry is exported through the VCC control plane and delivered to the learner’s local collector using a pod-scoped credential.

Decoder and rule

- A **decoder** extracts fields or interprets structure.
- A **rule** evaluates decoded information and can create an alert.

For JSON records, Wazuh can use its JSON decoder and dynamic fields. Custom rules should use a documented local rule-ID range and be tested with `wazuh-logtest` before deployment.

---

4. Raw events, decoded events and alerts

Consider this synthetic VCC record:

```
{
  "schema_version": "1.0",
  "event_time": "2026-08-01T09:18:07Z",
  "ingest_time": "2026-08-01T09:18:10Z",
  "pod_id": "pod-03",
  "event_type": "authentication",
  "action": "login",
  "outcome": "success",
  "user": "svc-backup",
  "source_ip": "203.0.113.44",
  "destination_service": "learner-api",
  "correlation_id": "corr-7d9f0a",
  "synthetic": true
}
```

Raw event

The complete JSON line written by the collector.

Decoded event

Wazuh identifies fields such as `pod_id`, `action`, `outcome`, `user` and `source_ip`.

Alert

A custom rule might alert when a successful login from a source follows repeated failures for the same account. The alert includes rule metadata and selected event fields.

The analyst should retain the distinction:

- the raw record came from the VCC feed;
- the decoder exposed fields;
- the rule made a detection decision;
- the analyst adds context and reaches a case conclusion.

---

5. Time in security investigations

Event time

When the source says the activity occurred.

Ingest time

When the platform received or indexed the record.

Processing or alert time

When a rule evaluated the record and created an alert.

These times can differ because of buffering, network delay, batch delivery or processing load.

Clock skew

Clock skew is a difference between system clocks. A host running three minutes fast can make a later action appear earlier than a preceding event on another system.

Time-zone handling

Prefer UTC for correlation. Preserve the source time zone when available and document any conversion.

Example

```
Application event time: 09:14:21Z
Collector received:      09:15:05Z
Indexed:                 09:15:09Z
Alert created:           09:15:10Z
```

The 44-second delivery delay is not automatically suspicious. It becomes important when building a precise sequence.

---

6. Data-quality failures

Failure	What the analyst may see	Risk to conclusion	First checks
Auditing disabled	Expected event never exists	“No evidence” mistaken for “no activity”	Source policy and test event

Failure	What the analyst may see	Risk to conclusion	First checks
Agent or collector disconnected	Events stop suddenly	Blind period around incident	Last heartbeat, queue and service status
Parser failure	Raw data exists but fields are empty	Searches and rules miss records	Raw message, decoder logs and schema version
Schema drift	Old fields disappear or change type	Rules silently stop matching	Compare recent and older samples
Clock skew	Events appear out of order	False timeline	Source time, NTP status and ingest time
Duplicate ingestion	Counts double or thresholds fire	Inflated scope and false alerts	Event IDs, hashes and collector paths
Dropped events	Missing records during high volume	Incomplete scope	Queue, rate limits and volume metrics
Retention expiry	Older events unavailable	Root cause cannot be reconstructed	Index age and archive policy
Wrong field type	Exact/range searches behave oddly	False negatives or misleading results	Index mapping
Truncation	Command, URI or payload is incomplete	Important context missing	Source limits and raw record
Redaction error	Sensitive data remains or useful context removed	Privacy exposure or lost evidence	Logging policy and sanitisation tests

7. Schema drift

Schema drift occurs when field names, nesting, formats or data types change.

Version 1:

```
{"source_ip":"203.0.113.44","user":"student03"}
```

Version 2:

```
{"source":{"ip":"203.0.113.44"},"identity":{"name":"student03"}}
```

A rule expecting `source_ip` may stop working even though records still arrive.

Controls

- include a `schema_version` field;
- validate required fields in the exporter;
- maintain sample fixtures for each supported version;
- test decoders and rules in CI;
- monitor field-population rates;
- reject or quarantine unsupported versions rather than silently misparse them.

---

8. Validating a telemetry pipeline

When an expected event is missing, work from source to destination.

Source

- Is the activity configured to generate a log?
- Can a safe test event be produced?
- Is the local record present?

Collector

- Is the service running?
- Does it have permission to read the source?
- Is its cursor or checkpoint advancing?

Transport

- Is authentication valid?
- Are TLS and network connections successful?
- Are queues full or retries increasing?

Parsing

- Is the raw event received?
- Are required fields extracted?
- Did the schema change?

Indexing

- Is the target index writable and searchable?
- Are mappings correct?
- Is the selected time range based on the correct timestamp?

Detection

- Does the rule load successfully?
- Do the exact fields and values satisfy the condition?
- Is the threshold window correct?
- Was the event excluded or suppressed?

Dashboard

- Are filters hiding the record?
- Is the correct index pattern or data view selected?
- Is the browser displaying the intended time zone?

---

9. Worked example — alert disappeared after application update

Symptom

A rule that detects failed logins stops generating alerts after an application release.

Incorrect conclusion

> Failed login attempts have stopped.

Investigation

1. The application still writes JSON records.
2. The collector continues receiving current events.
3. Raw records now use `authentication.result` instead of `outcome`.
4. The Wazuh rule expects `outcome` equal to `failure`.
5. Indexed documents contain the new nested field, but the old field is empty.

Finding

The monitoring gap was caused by schema drift. Failed logins continued, but the existing rule no longer matched them.

Remediation

- update and test the decoder/rule;
- version the schema;
- add a CI fixture for both transition formats;

- monitor the population rate of required authentication fields;
- document the blind period.

---

10. Detection context and ATT&CK

MITRE ATT&CK helps describe adversary behaviour, but an ATT&CK technique label does not replace detection logic.

A useful detection design states:

1. **Threat hypothesis:** What behaviour are we concerned about?
2. **Required telemetry:** Which sources and fields can show it?
3. **Analytic:** What sequence, relationship or threshold should be evaluated?
4. **Expected benign causes:** Which authorised behaviours may look similar?
5. **Validation:** How will synthetic tests show that the detection works?
6. **Tuning record:** What changes were made and what coverage might be lost?

Current ATT&CK teaching should use Detection Strategies, Analytics and Data Components rather than treating a technique ID as a complete detection.

---

11. Common mistakes

Believing structured JSON is automatically correct

JSON can contain wrong timestamps, empty fields, unexpected types or attacker-controlled values.

Searching only the friendly message

Use structured fields and inspect the raw record. Friendly descriptions may omit important context.

Treating source IP as unquestionable

Proxies, NAT, forwarded headers and collection architecture affect which address is recorded.

Ignoring ingest delay

A delayed event can appear after an analyst has already closed a case.

Tuning by deleting noisy data

Noise can expose misconfiguration, stale credentials or operational problems. Understand the cause before excluding evidence.

Exposing internal services

The Wazuh indexer and server API should not be publicly exposed merely for convenience. Use the local dashboard and approved internal connections.

---

12. Guided exercise — pipeline failure classification

For each situation, identify the pipeline stage and likely investigative effect.

1. A Windows host never generated Event 4688 because process-creation auditing was disabled.
2. The VCC collector certificate expired and polling returned HTTP 401/403.
3. JSON records arrive, but `pod_id` is missing after a release.
4. The same file is read by two collectors.
5. A query searches the last hour using ingest time while the incident occurred two hours earlier and arrived late.
6. The index retained only seven days, but the suspected access occurred ten days ago.

Then write one validation step for each situation.

---

13. Review questions

1. What is the difference between log generation and collection?
2. Why should raw records remain available where policy permits?
3. Explain normalisation and one risk of over-normalising.
4. What is schema drift?
5. Why can event time and ingest time differ?
6. What does a decoder do in Wazuh?
7. What does the Wazuh indexer do?
8. Why should the indexer not be exposed publicly in the student profile?
9. Give three reasons why “no alert” does not prove “no attack.”
10. What six elements belong in a detection design?

References

- NIST SP 800-92 and SP 800-92 Rev. 1 Initial Public Draft.

- Wazuh official documentation: architecture, log collection, JSON decoder, custom rules, WQL and Docker deployment.
- OpenSearch official Query DSL and field-mapping documentation.
- MITRE ATT&CK v19.1 Detection Strategies, Analytics and Data Components.
- Microsoft Sysmon and Windows auditing documentation.
- AWS CloudTrail documentation.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.

Source: docs/secops-foundations/04-incident-response-and-playbook-development.md

Module 4 — Incident Response and Playbook Development

> **NeoLabs SOC Level 1 principle:** Incident response is not only what happens after an alert. Preparation, governance, logging, access control, communication and recovery readiness determine whether the organisation can respond effectively.

Module version: 0.2-draft
Review baseline: 1 August 2026
Expected study time: 5–7 hours

Learning objectives

By the end of this module, a learner should be able to:

- explain the relationship between cybersecurity risk management and incident response;
- describe the current NIST SP 800-61 Rev. 3 approach using the six CSF 2.0 Functions;
- compare current NIST framing with older preparation/detection/containment/eradication/recovery/lessons models without confusing them;
- distinguish an alert, cybersecurity event, declared incident and crisis;
- explain the roles of Level 1 analysts, incident responders, incident commanders, system owners and business stakeholders;
- create a practical playbook with triggers, evidence requirements, decision points, escalation and recovery checks;
- distinguish containment, eradication and recovery actions;
- explain why response actions require authority and evidence preservation;
- use an incident report and evidence log during a synthetic VCC scenario.

---

1. What incident response means

Cybersecurity incident response is the coordinated capability used to prepare for, detect, analyse, contain, address, recover from and learn from cybersecurity incidents.

It includes more than technical commands. Effective response depends on:

- policy and authority;
- current asset and identity information;

- useful logging;
- trained people;
- communication paths;
- tested backups;
- legal, privacy and contractual awareness;
- technical containment and recovery procedures;
- improvement after the incident.

Plain-language explanation

Incident response is how an organisation answers:

1. What happened?
2. Is it actually a cybersecurity incident?
3. What is affected?
4. What must be protected immediately?
5. Which evidence must be preserved?
6. Who is authorised to decide and act?
7. How do we remove or reduce the threat?
8. How do we restore safe operations?
9. What must change afterward?

---

2. Current NIST framing

NIST SP 800-61 Rev. 3, finalised in April 2025, aligns incident response with the six NIST Cybersecurity Framework 2.0 Functions:

```
GOVERN
IDENTIFY
PROTECT
DETECT
RESPOND
RECOVER
```

The six Functions are not simply six chronological boxes. Govern, Identify and Protect help prepare the organisation and reduce incident likelihood or impact. Detect, Respond and Recover contain the core operational incident-response activity, while lessons and changes feed back into all Functions.

2.1 Govern

Govern establishes direction, accountability and risk decisions.

Incident-response examples:

- policy and incident definitions;
- executive sponsorship;
- roles and authorities;
- legal, regulatory and contractual requirements;
- third-party responsibilities;
- escalation thresholds;
- risk tolerance;
- reporting and communication governance.

SOC Level 1 relevance

The Level 1 analyst follows approved definitions and escalation paths rather than inventing severity or response authority during an alert.

2.2 Identify

Identify develops understanding of assets, services, risks, dependencies and vulnerabilities.

Incident-response examples:

- asset inventory;
- data classification;
- identity and privilege inventory;
- dependency maps;
- vulnerability information;
- risk assessment;
- critical business-service identification.

SOC Level 1 relevance

An unusual login to an ordinary test account and an unusual login to a critical administrator account may require different priority because the identity and business context differ.

2.3 Protect

Protect applies safeguards to reduce the likelihood and impact of adverse events.

Incident-response examples:

- access control and MFA;
- secure configuration;
- patching;
- awareness training;

- data protection;
- resilient architecture;
- backup protection;
- logging and monitoring preparation.

SOC Level 1 relevance

Protection controls create the evidence and boundaries used during triage. Weak logging or shared credentials reduce the analyst’s ability to reach a defensible conclusion.

2.4 Detect

Detect identifies and analyses possible cybersecurity events.

Examples:

- event collection;
- SIEM rules and analytics;
- anomalous activity review;
- monitoring for control failure;
- alert validation;
- initial analysis and incident determination.

SOC Level 1 relevance

This is where much L1 triage work occurs: validate the alert, review evidence, search related activity, classify and escalate.

2.5 Respond

Respond contains actions taken after an incident is detected and declared.

Examples:

- incident management;
- analysis and scope expansion;
- internal and external communication;
- containment;
- mitigation;
- evidence handling;
- coordinated decision-making.

SOC Level 1 relevance

L1 analysts usually document and escalate. They may perform only the response actions explicitly authorised by a playbook.

2.6 Recover

Recover restores affected assets, services and operations and communicates recovery status.

Examples:

- restoration from trusted backups;
- rebuilding systems;
- credential reset;
- verification of safe service operation;
- monitoring after restoration;
- stakeholder communication;
- recovery improvement.

SOC Level 1 relevance

Analysts may monitor restored systems and confirm that alert patterns, telemetry and expected behaviour return to normal.

---

3. Older incident-handling phase models

Many textbooks and older procedures describe a sequence such as:

```
Preparation
  → Detection and Analysis
  → Containment
  → Eradication
  → Recovery
  → Lessons Learned
```

NIST SP 800-61 Rev. 2 used a four-phase life cycle that grouped containment, eradication and recovery together and included post-incident activity. That publication was superseded by Rev. 3 in April 2025.

The older terms remain operationally useful. Do not say they are “wrong,” but do not present them as the current NIST Rev. 3 structure.

Practical mapping

Traditional term	Current CSF-aligned location
Preparation	Govern, Identify and Protect, with preparation across all Functions
Detection and analysis	Detect and parts of Respond

Traditional term	Current CSF-aligned location
Containment and eradication	Respond
Recovery	Recover
Lessons learned	Feedback into Govern, Identify, Protect, Detect, Respond and Recover

4. Alert, event, incident and crisis

Security event

A security-relevant occurrence recorded by a system or reported by a person.

Alert

A detection-generated notification that an event or sequence deserves review.

Cybersecurity incident

An occurrence that meets the organisation’s declared incident criteria because it actually or imminently threatens systems, information, policy or operations and requires coordinated response.

Major incident or crisis

An incident with significant operational, financial, safety, legal, customer or reputational impact that requires executive coordination beyond the normal SOC process.

Important distinction

The SIEM can generate an alert. An authorised organisational process declares an incident. A Level 1 analyst may recommend declaration or escalate evidence, but the exact authority depends on policy.

5. Incident-response roles

5.1 SOC Level 1 analyst

- validates alert evidence;
- gathers initial context;
- identifies affected entities;

- records queries and a preliminary timeline;
- follows the relevant playbook;
- escalates on defined triggers;
- avoids unauthorised containment.

5.2 SOC Level 2 or incident responder

- expands scope;
- correlates additional sources;
- validates compromise and impact;
- coordinates authorised containment;
- preserves and requests deeper evidence;
- develops eradication and recovery recommendations.

5.3 Incident commander

The **incident commander** coordinates the response, maintains priorities, assigns owners, manages decisions and ensures communication. The commander does not need to perform every technical investigation personally.

5.4 System and service owners

They explain expected operation, approve or perform service-impacting actions and assess business consequences.

5.5 IT, cloud and application teams

They may isolate hosts, revoke credentials, change firewall or IAM policies, deploy fixes, restore services and validate normal operation.

5.6 Legal, privacy, communications and management

These stakeholders assess obligations, sensitive-data exposure, notification, public communication and business risk.

VCC training boundary

Mentors and operators retain containment authority over VCC infrastructure. Interns investigate the approved student-facing evidence and make recommendations.

---

6. Incident classification and declaration

An organisation should define incident categories and thresholds before an event occurs.

Possible factors:

- affected asset criticality;
- privilege of affected identities;
- confidentiality, integrity or availability impact;
- number of affected users or systems;
- persistence or lateral movement;
- data access or exfiltration evidence;
- legal or contractual significance;
- operational disruption;
- confidence in the evidence;
- whether the condition remains active.

Severity and confidence remain separate

A suspected compromise of a privileged cloud account may be potentially critical even when initial confidence is medium. Urgent escalation can be justified by risk while investigation continues.

---

7. Containment, eradication and recovery

7.1 Containment

Containment limits the spread, activity or impact of the incident.

Examples:

- revoke a compromised session;
- disable or restrict an account;
- isolate an endpoint;
- block a known malicious destination;
- remove an exposed service from public access;
- temporarily disable a vulnerable function;
- preserve a system before rebuilding.

Containment caution

Containment can disrupt business, alert an adversary or destroy volatile evidence. It must be proportionate and authorised.

7.2 Eradication

Eradication removes the cause or remaining threat.

Examples:

- remove malicious persistence;
- patch the exploited vulnerability;
- delete unauthorised accounts after evidence preservation;
- rotate exposed secrets;
- remove a malicious application or configuration;
- close the exploited access path.

Eradication is not only deleting a suspicious file. The root cause and all affected scope must be addressed.

7.3 Recovery

Recovery restores trustworthy operation.

Examples:

- rebuild from a trusted image;
- restore clean data from backup;
- verify patched configuration;
- re-enable services gradually;
- monitor closely for recurrence;
- confirm logging and detection are working;
- communicate service status.

Recovery validation

A service being reachable does not prove recovery is complete. Validate identity, integrity, security controls, telemetry and expected business functions.

---

8. Evidence preservation during response

Response and evidence collection can conflict. For example, restarting a system may restore service but remove volatile evidence.

Level 1 analysts should:

- record important event IDs and timestamps;
- preserve approved exports before filters or retention change;
- document who performed each action and when;
- distinguish source evidence from screenshots;

- avoid modifying the affected system without authority;
- record visibility gaps created by containment;
- protect credentials and personal information.

The NeoLabs evidence log is a training record, not a substitute for formal forensic chain-of-custody procedures where those are required.

---

9. What a playbook is

A **playbook** is a documented, repeatable response guide for a particular incident type or trigger.

A playbook supports consistent action under pressure. It does not remove analyst judgment. It should define decision points, authority and stop conditions rather than list commands without context.

Playbook versus runbook

Terminology differs, but a useful distinction is:

- **Playbook:** broader incident strategy, decisions, roles and coordinated actions.
- **Runbook:** detailed procedure for a specific repeatable technical task.

Example:

- Account-takeover playbook: triage, declaration, communication, containment and recovery decisions.
- Session-revocation runbook: exact approved steps to revoke sessions in one identity platform.

---

10. Required playbook sections

10.1 Purpose and scope

State:

- incident type;
- systems and environments covered;
- authorised users;
- exclusions;
- linked policies.

10.2 Trigger

Define what starts the playbook:

- SIEM alert;
- user report;
- threat-intelligence notification;
- monitoring outage;
- cloud-provider alert;
- confirmed vulnerability exposure.

A trigger starts investigation. It does not necessarily confirm an incident.

10.3 Required evidence

List the minimum sources and fields needed.

Example for account takeover:

- failed and successful authentication;
- MFA result;
- session creation and revocation;
- source and device context;
- password or email changes;
- sensitive actions after login;
- account owner confirmation;
- telemetry-health status.

10.4 Initial L1 actions

Keep them bounded and reproducible:

1. validate the underlying event;
2. establish absolute UTC range;
3. identify account, source, asset and session;
4. search preceding and following activity;
5. check known maintenance and account role;
6. document observations and queries;
7. classify and escalate on defined conditions.

10.5 Escalation criteria

Examples:

- privileged account affected;
- successful access after suspicious failures;

- sensitive account or data change;
- multiple accounts targeted;
- active session remains valid;
- missing telemetry after the success;
- possible cross-pod data exposure;
- response action needed;
- uncertainty cannot be resolved within L1 authority.

10.6 Containment options

Each option should state:

- authorising role;
- operational impact;
- evidence consideration;
- verification step;
- rollback or recovery dependency.

10.7 Eradication and recovery

Define how root cause is addressed and how trustworthy operation is confirmed.

10.8 Communication

State:

- who receives initial escalation;
- who owns stakeholder updates;
- approved communication channels;
- required update frequency;
- what information must not be placed in public channels.

10.9 Closure and improvement

Require:

- final classification;
- timeline;
- affected scope;
- actions and owners;
- evidence references;
- unresolved risks;
- detection and logging improvements;
- playbook revision if needed.

11. Worked playbook — suspected account takeover

Purpose

Investigate authentication failures followed by a successful login and sensitive account activity in an authorised synthetic VCC scenario.

Trigger

A Wazuh rule reports success after repeated failures for the same account.

L1 evidence requirements

- raw failure and success records;
- user and source address;
- authentication method;
- session ID;
- post-login actions;
- other accounts targeted by the source;
- telemetry-health events;
- approved maintenance or account-owner context.

L1 workflow

1. Set an absolute UTC range beginning 15 minutes before the first failure.
2. Validate distinct event IDs and exclude duplicate ingestion.
3. Count failures for the account and source.
4. Confirm whether the success came from the same source.
5. Pivot to the created session.
6. Identify account, authorisation and application events.
7. Search for other targeted users.
8. Identify telemetry gaps.
9. write observation, interpretation and conclusion.
10. Escalate if success, sensitive action, privileged identity, multi-account targeting or visibility gap is present.

Potential containment recommendations

An authorised responder may:

- revoke the suspicious session;
- reset or rotate credentials;
- require fresh MFA enrolment when appropriate;
- temporarily restrict the account;
- block the source when justified and operationally safe;

- preserve relevant identity and application logs.

The intern documents recommendations and observed scenario-controller actions but does not directly administer VCC identity systems.

Eradication questions

- How were credentials obtained or guessed?
- Are other credentials or sessions affected?
- Was a weak authentication or recovery flow involved?
- Was the account used to create persistence?
- Which control or vulnerability enabled access?

Recovery checks

- suspicious sessions revoked;
- credentials rotated;
- expected owner access restored;
- sensitive account values verified;
- monitoring and logging healthy;
- no repeat activity during the defined observation period;
- detection and rate-limit controls reviewed.

---

12. Playbook anti-patterns

A list of destructive commands

A playbook should not assume every alert deserves isolation, deletion or blocking.

No authority information

Analysts need to know who can approve each action.

No evidence requirements

Without evidence criteria, the team may act on an alert title alone.

One path with no decision points

Real incidents differ. Include conditions for close, continue, escalate and declare.

No recovery validation

“Service restored” is incomplete without trust and monitoring checks.

Secrets inside the playbook

Use secret references, never actual passwords, tokens or private endpoints.

No owner or review date

Outdated playbooks can be worse than no playbook when tools and systems change.

---

13. Incident communication

Good incident communication is:

- factual;
- time-stamped;
- clear about uncertainty;
- appropriate for the audience;
- limited to approved recipients;
- consistent with the case record.

Technical update example

> At 09:18:07Z, synthetic authentication telemetry recorded a successful login for `svc-backup` from the source that produced five earlier failures. The resulting session later accessed a profile and attempted cross-pod evidence access, which the application denied. A process-telemetry gap began before the success, so endpoint activity cannot currently be confirmed or excluded. Classification: suspicious synthetic incident; confidence medium-high. Session revocation is recorded. Further owner and telemetry review is requested.

Poor update

> Hacker entered the system and tried to steal everything.

The poor update overstates identity, intent and impact.

---

14. Post-incident improvement

Improvement should consider:

- logging fields and source coverage;
- detection logic and false-positive patterns;
- playbook clarity;
- escalation delay;
- authority and communication gaps;
- backup and recovery performance;
- identity and access controls;
- asset and dependency records;
- training needs;
- third-party coordination.

A lessons-learned review should not become a blame session. Focus on system and process improvement while preserving accountability.

---

15. Guided exercise

Using Practice Lab 01:

1. decide whether the evidence meets the training definition of an incident;
2. map observed actions to Detect, Respond and Recover;
3. identify which Govern, Identify and Protect activities should have existed before the alert;
4. draft an account-takeover playbook using the template below;
5. identify three actions outside L1 authority;
6. write one technical update and one management-friendly summary;
7. list five lessons or control improvements.

Playbook draft template

```
Title:
Owner:
Version/review date:
Purpose and scope:
Trigger:
Required evidence:
Initial L1 actions:
Decision points:
Escalation criteria:
Incident declaration authority:
Containment options and authorisers:
Eradication requirements:
Recovery checks:
Communication plan:
Evidence and reporting requirements:
Closure criteria:
Post-incident review:
References:
```

---

16. Review questions

1. How does NIST SP 800-61 Rev. 3 organise incident-response recommendations?
2. Why is the older six-step model still useful but not the current NIST structure?
3. Who normally declares an incident?
4. What is the difference between containment and eradication?
5. Why can containment damage evidence?
6. What makes a playbook different from a simple command list?
7. Name five required playbook sections.
8. Why should recovery include logging and monitoring validation?
9. What information belongs in an incident update?
10. How do lessons feed back into all six CSF Functions?

Authoritative references

- NIST SP 800-61 Rev. 3, *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*.
- NIST Cybersecurity Framework 2.0.
- NIST SP 800-61 Rev. 2, withdrawn and superseded, for historical incident-handling phase terminology.
- CISA, *Federal Government Cybersecurity Incident and Vulnerability Response Playbooks*, used as public playbook structure context rather than as VCC policy.
- NeoLabs evidence, incident-report and query-journal templates.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.

Source: docs/secops-foundations/05-windows-and-sysmon-investigation.md

Module 5 — Windows and Sysmon Investigation

Learning objectives

By the end of this module, a SOC Level 1 analyst should be able to:

- distinguish Windows security, system, PowerShell and Sysmon telemetry;
- build a short authentication and process timeline;
- recognise common high-value event relationships;
- separate suspicious behaviour from confirmed malicious activity;
- document evidence without exposing credentials or private infrastructure.

1. Windows evidence sources

A Windows investigation rarely depends on one event. The analyst correlates records that answer different questions.

Source	Typical questions
Security log	Who signed in, from where, with which account and logon type?
System log	Did a service, driver or operating-system component fail or change?
PowerShell logs	What commands or script blocks were executed?
Microsoft Defender logs	Was malware, a potentially unwanted application or suspicious behaviour detected?
Sysmon	Which process, network, file, registry or image-loading activity occurred?
Application logs	Did the affected business application report authentication or execution errors?

No single event source is complete. Missing telemetry must be recorded as an investigation limitation rather than silently treated as evidence that nothing happened.

2. Authentication timeline

Begin with the account, device and time window supplied in the alert. Expand the window enough to see activity immediately before and after the trigger.

Important Windows authentication concepts include:

- successful and failed logons;
- interactive, network, service, batch and remote-interactive logon types;

- source workstation or network address;
- privileged-logon indicators;
- account lockout and password-change activity;
- logoff or session termination.

A useful timeline might show:

```
09:12:03  repeated failed network logons for trainee-a
09:13:20  successful remote-interactive logon for trainee-a
09:13:42  privileged process created in the new session
09:14:10  outbound connection to an unusual destination
09:16:02  session terminated
```

The timeline does not prove compromise by itself. It establishes sequence, scope and the next questions to test.

3. Process investigation

For each suspicious process, capture:

- process image and full path;
- parent process and parent path;
- command line;
- user and session identifier;
- hash when available;
- signature or publisher information when available;
- start time;
- network, file or registry activity linked to the process.

A process name is weak evidence. Attackers can rename files, while legitimate tools can appear suspicious when used by administrators. Parent-child relationships and command-line context are usually more useful.

Examples of questions:

- Was the process launched by the expected parent?
- Did an Office application, browser or script host create a command shell unexpectedly?
- Was the executable started from a user-writable or temporary path?
- Did the command line contain encoded content, credential access attempts or unusual download behaviour?
- Did the process create persistence or connect externally?

4. Sysmon relationships

Sysmon records detailed activity, but the analyst still needs correlation. High-value relationships include:

```
process creation
  -> network connection
  -> file creation
  -> registry change
  -> child process
```

Use stable identifiers, such as process GUIDs, when they are available. Process IDs can be reused after a process ends and should not be treated as globally unique.

Process creation

Capture the image, command line, parent, user, hashes and process GUID. Compare the path and parent relationship with the host's normal pattern.

Network connection

Link the connection back to the process. Record destination IP or hostname, port, protocol, connection direction and whether the destination is expected for that application.

File creation and modification

Record the path, creating process and timestamp. Files in temporary directories, startup folders or unusual user-profile locations deserve context, not automatic conviction.

Registry activity

Focus on persistence locations, security-setting changes and application-specific configuration. A registry modification is meaningful only when the key, value, actor and timing are understood.

5. PowerShell review

PowerShell is a legitimate administrative tool and a common attack surface. Review:

- script-block content where available;
- module and engine events;
- parent process;
- user and session;
- encoded or obfuscated content;
- download, execution-policy or security-control changes;
- follow-on process and network activity.

Do not paste sensitive command output into public repositories. Redact tokens, passwords, private URLs and identifying student information.

6. Triage decision

Use an evidence-based disposition:

Benign or expected

The activity matches an approved administrative or learning task, the account and device are expected, and no contradictory evidence is present.

Suspicious — investigate further

The activity is unusual or incomplete, but available evidence does not yet establish malicious intent or impact.

Likely malicious — escalate

Multiple independent signals support unauthorised activity, such as an unexpected remote logon followed by suspicious process execution and external communication.

False positive or detection-quality issue

The event is real, but the rule or enrichment produced misleading severity or context. Preserve the evidence and recommend a safe tuning change.

7. Evidence checklist

For every Windows investigation, record:

1. alert identifier and trigger time;
2. affected host and account;
3. investigation time range and timezone;
4. relevant event sources;
5. authentication sequence;
6. process tree or key parent-child relationship;
7. network, file and registry evidence;
8. missing evidence and limitations;
9. disposition and confidence;
10. recommended containment or escalation.

8. Practice exercise

Using a synthetic dataset, identify a failed-login sequence followed by a successful remote-interactive logon. Build a five-event timeline, identify the first process created in the session and decide whether the evidence supports benign activity, suspicious activity or escalation.

Submit the result using the evidence-log, query-journal and incident-report templates. Do not include credentials, private pod addresses or mentor-only ground truth.

Source: docs/secops-foundations/06-linux-web-and-cloud-log-investigation.md

Module 6 — Linux, Web and Cloud Log Investigation

Learning objectives

By the end of this module, a SOC Level 1 analyst should be able to:

- identify useful Linux, web-server, application and cloud-audit sources;
- correlate authentication, process, network and application events;
- recognise evidence-quality problems caused by time, parsing or missing fields;
- build a cross-source timeline without assuming that every unusual event is malicious;
- escalate findings with clear scope and limitations.

1. Linux evidence sources

Common Linux sources include:

Source	Typical evidence
`journald` or system logs	service starts, failures, kernel and authentication messages
Authentication logs	SSH, sudo, account and session activity
Audit logs	system calls, file access, process execution and policy events
Shell history	limited user-command context when available and trustworthy
Package-manager logs	software installation, upgrade and removal
Cron and systemd timers	scheduled execution and persistence
Web-server logs	requests, response codes, user agents and source addresses
Application logs	authentication, errors, business events and API activity
Database logs	connection, authentication and query-related events
Cloud audit logs	API calls, identity, source address and resource changes

Shell history is not a complete or tamper-resistant record. Treat it as supporting context, not as the only evidence of execution.

2. Linux authentication triage

For SSH or privilege-related alerts, capture:

- account name;
- source address;
- authentication method;
- success or failure;
- session start and end;
- `sudo` or privilege-transition activity;
- processes created in the session;
- changes to accounts, keys or authentication configuration.

A repeated-failure sequence followed by a success is important only when combined with context. Questions include:

- Is the source expected for the learner or administrator?
- Was the account active at that time?
- Did the successful session perform unusual activity?
- Were authorised maintenance or lab exercises scheduled?
- Did the source attempt multiple accounts or hosts?

3. Process and persistence review

Review execution from temporary, user-writable and hidden locations carefully. Record the process, parent, user, command line and linked network activity.

Common persistence locations to examine in an authorised investigation include:

- user and system cron entries;
- systemd services and timers;
- shell profile files;
- SSH `authorized_keys`;
- application startup hooks;
- container restart policies;
- cloud-init or deployment scripts.

Presence in one of these locations is not automatically malicious. Package installers and administrators use them legitimately. The analyst must determine who created the entry, when, why and what it executes.

4. Web-server investigation

Web access logs often provide:

```
source address
request time
HTTP method
request path
status code
response size
referrer
user agent
request duration
```

Application and reverse-proxy logs may add authenticated user, request ID, route name, upstream status and error details.

High-value patterns

Investigate patterns such as:

- many failed logins followed by a success;
- repeated requests to non-existent administrative paths;
- unusual methods or content types;
- path traversal strings or unexpected encoded paths;
- a request producing an application exception;
- upload activity followed by execution-like behaviour;
- a single request ID appearing across proxy, application and database logs.

A suspicious request does not prove successful exploitation. Confirm the response, application behaviour, downstream logs and resulting system changes.

5. Application and database correlation

Application logs provide business context that infrastructure logs may lack. Use correlation identifiers where available.

Example timeline:

```
10:21:03  reverse proxy accepts POST /login from source A
10:21:03  application records failed authentication for user trainee-b
10:21:04  database records expected account lookup
10:22:18  reverse proxy accepts another POST /login from source A
10:22:18  application records successful authentication
10:22:24  application records export request for an unusual resource
```

Confirm that clocks and timezones align before ordering events. A two-minute clock difference can create a misleading sequence.

6. Cloud audit investigation

Cloud audit records commonly include:

- identity or role;
- API action;
- resource;
- source address and user agent;
- region;
- success or error result;
- request parameters;
- request or event identifier.

For an unexpected cloud action, determine:

1. which identity performed it;
2. how the identity authenticated;
3. whether the source and user agent are expected;
4. which resources were affected;
5. whether related actions occurred before or after it;
6. whether the action matches an approved lab scenario.

Never copy live cloud keys, session tokens or unredacted account identifiers into the public toolkit.

7. Data-quality checks

Before making a conclusion, check:

- timestamp format and timezone;
- host and source identity;
- duplicate events;
- parser or decoder errors;
- missing fields;
- delayed ingestion;
- inconsistent field names across sources;
- truncation of commands, URLs or messages;
- whether logs are synthetic, sanitised or live.

A data-quality defect can be the root cause of a misleading alert. Record it separately from the security disposition.

8. Correlation method

Use a repeatable method:

1. define the alert entity and time window;

2. collect the original event;
3. pivot by account, host, source address, process, request ID or resource;
4. normalise time and field names;
5. place events in chronological order;
6. test benign and malicious explanations;
7. identify gaps;
8. assign disposition and confidence;
9. document escalation or tuning recommendations.

9. Practice exercise

Investigate a synthetic web-login alert that includes Linux authentication events, reverse-proxy requests and application records. Determine whether the successful login is connected to the earlier failures, identify any follow-on activity and produce a short evidence-based escalation note.

The submission must state which logs were available, which were missing and how those limitations affected confidence.

Source: docs/secops-foundations/07-wazuh-alert-investigation-and-tuning.md

Module 7 — Wazuh Alert Investigation and Safe Tuning

Learning objectives

By the end of this module, a SOC Level 1 analyst should be able to:

- move from a Wazuh alert to the underlying evidence;
- identify decoder, rule, agent and event fields that affect interpretation;
- use filters and pivots to build an investigation timeline;
- distinguish alert disposition from rule-quality feedback;
- propose safe tuning without hiding meaningful security activity.

1. Start with the alert record

Capture the original alert before changing filters or running broader searches. Record:

- alert identifier;
- rule ID, level and description;
- agent or source identity;
- event timestamp and ingestion timestamp;
- decoder or integration name;
- full original event when available;
- relevant structured fields;
- scenario or assignment context.

The rule description is a hypothesis, not the final incident conclusion.

2. Understand the Wazuh processing path

A simplified path is:

```
source event
  -> collection
  -> decoding and field extraction
  -> rule evaluation
  -> alert creation
  -> indexing
  -> dashboard search and visualisation
```

An investigation can fail at any stage. A missing alert may mean:

- the source did not generate the event;
- the collector did not receive it;

- the event was malformed;
- the decoder did not extract expected fields;
- the rule conditions did not match;
- the alert was indexed under an unexpected time or field;
- the dashboard filter excluded it.

Do not jump directly to changing a rule before checking the earlier stages.

3. Alert-to-evidence pivots

Useful pivots include:

- agent ID or host name;
- account;
- source and destination address;
- process image, process ID or process GUID;
- rule ID;
- event ID;
- request or correlation ID;
- file path or hash;
- scenario ID;
- pod ID supplied by the server-authorised telemetry stream.

Begin with a narrow time window and expand only when needed. Preserve the timezone in the query journal.

4. Rule fields and interpretation

When reviewing an alert, separate:

Source facts

Values that came from the original event, such as account, process, source address, path or result.

Decoder output

Fields extracted or normalised by Wazuh. A decoder error can mislabel, omit or split data.

Rule metadata

Rule ID, level, groups, description, frequency and relationship to parent or child rules.

Enrichment

Information added from inventories, threat intelligence or lookup data. Enrichment can be missing, stale or incorrect and must not replace source evidence.

5. Investigation workflow

1. Save the original alert.
2. Confirm the affected entity and timestamp.
3. Inspect the raw event and decoded fields.
4. Search for the same entity immediately before and after the alert.
5. Pivot to related authentication, process, network, file or application activity.
6. Compare the behaviour with approved lab activity.
7. Record gaps and data-quality concerns.
8. Assign a disposition and confidence.
9. Escalate, close or recommend tuning.

6. Disposition and rule quality are different

An alert can be a true positive event but still have poor severity. It can also be a false positive caused by a decoder or rule issue.

Use two separate conclusions:

Security disposition

- expected activity;
- suspicious activity requiring more evidence;
- likely malicious activity requiring escalation;
- confirmed authorised simulation.

Detection-quality assessment

- rule behaved correctly;
- severity too high or too low;
- missing context or enrichment;
- duplicate or noisy alert;
- decoder issue;
- field-mapping issue;
- rule logic requires review.

This separation prevents analysts from weakening a rule simply because one event was benign.

7. Safe tuning principles

A tuning proposal should be narrow, testable and reversible.

Good proposals may:

- add a condition based on a stable, well-understood field;
- reduce duplicate alerts while preserving the first and most important signal;
- correct a decoder or field mapping;
- add context to the alert description;
- change severity based on verified impact or privilege;
- create an allow-list limited to an approved account, host, process path and time-bound use case.

Avoid broad exclusions such as:

```
ignore all PowerShell
ignore this source network
ignore all administrator logons
ignore the entire rule group
```

Broad suppression can hide unrelated malicious activity.

8. Tuning proposal template

Document:

- rule ID and current behaviour;
- evidence showing the problem;
- proposed change;
- expected effect;
- possible blind spots;
- test dataset;
- rollback method;
- reviewer and approval status.

Students propose tuning. They do not change shared VCC control-plane rules, mentor dashboards or production detections.

9. Validation cases

Every proposed change should be tested against:

1. the benign event that created noise;
2. a similar malicious or suspicious event that must still alert;
3. unrelated events that should remain unchanged;
4. malformed or missing-field events;

5. duplicate events;
6. the expected rule level and groups.

Use synthetic or sanitised fixtures. Do not copy live incidents into this public repository.

10. Escalation package

A strong Wazuh escalation contains:

- alert and rule details;
- raw event evidence;
- timeline of related activity;
- affected entities;
- analyst queries;
- security disposition and confidence;
- detection-quality observation;
- recommended containment or next action;
- limitations.

11. Practice exercise

Investigate a synthetic Wazuh authentication alert that fires repeatedly for the same sequence. Determine the security disposition, identify whether duplication is caused by repeated source events or rule behaviour, and write a tuning proposal that reduces noise without suppressing the successful-login event.

Source: docs/secops-foundations/08-case-management-reporting-and-capstone.md

Module 8 — Case Management, Reporting and SOC Capstone

Learning objectives

By the end of this module, a SOC Level 1 analyst should be able to:

- maintain an investigation record that another analyst can reproduce;
- distinguish facts, assumptions, analysis and recommendations;
- communicate severity, confidence and business impact clearly;
- hand off a case without losing evidence or context;
- complete a defensive capstone using the NeoLabs templates.

1. The case record

A case is more than an alert screenshot. It is the organised record of what was reported, what was examined, what was found, what remains unknown and what should happen next.

A minimum case record includes:

- case or issue identifier;
- title and current status;
- owner and reviewers;
- alert source;
- affected accounts, hosts, applications or resources;
- detection, triage and escalation timestamps;
- evidence register;
- query journal;
- timeline;
- disposition and confidence;
- recommended actions;
- limitations and unanswered questions.

2. Facts, analysis and assumptions

Keep these categories separate.

Fact

A directly observed and preserved item, such as:

```
The authentication event records a successful remote-interactive logon for trainee-a at 09:13:20 UTC.
```

Analysis

An interpretation supported by facts:

```
The success occurred 77 seconds after repeated failures from the same synthetic source, increasing the
likelihood that the events are related.
```

Assumption

A statement not yet proven:

```
The source may represent the same operator throughout the sequence.
```

Recommendation

A proposed action:

```
Escalate for review of the account's approved activity and preserve the related process telemetry.
```

Mixing these categories makes reports difficult to trust.

3. Severity and confidence

Severity describes the potential or observed effect. Confidence describes how strongly the evidence supports the conclusion.

Example:

```
Severity: High
Confidence: Medium
Reason: The account obtained a privileged session and launched an unusual process, but endpoint network
telemetry is incomplete.
```

Do not raise confidence merely because an alert has a high rule level. Do not lower severity solely because the dataset is synthetic; assess the scenario as instructed while clearly marking the evidence as synthetic.

4. Timeline construction

A useful timeline:

- uses one timezone;
- orders events chronologically;
- identifies the source of each entry;
- distinguishes event time from ingestion time when relevant;
- avoids duplicate entries;
- includes the analyst's key pivots;
- marks gaps or uncertain ordering.

Time (UTC)	Source	Entity	Event	Significance
09:12:03	Windows Security	trainee-a	Failed logon	Start of repeated failures
09:13:20	Windows Security	trainee-a	Successful remote logon	First success in sequence
09:13:42	Sysmon	host-01	Process creation	Follow-on execution

5. Evidence handling

The evidence log should record:

- evidence ID;
- source and collection method;
- timestamp;
- file, event or screenshot reference;
- checksum when a file is submitted;
- analyst notes;
- redactions;
- storage location.

Do not alter original evidence to make a report look cleaner. Create a redacted copy and retain the original only in the approved private location.

The public toolkit and student submissions must not contain:

- passwords or tokens;
- private keys or certificates;
- private pod URLs;
- AWS account details;
- unredacted personal data;
- mentor ground truth;
- production evidence.

6. Query journal

A query journal makes the investigation reproducible. For each query, record:

- tool and data source;
- exact query or command;
- time range and timezone;
- purpose;
- result count;
- useful findings;
- limitations;
- next pivot.

A query that returns no results is still relevant when the source, time range and field assumptions are documented.

7. Handoff and escalation

A handoff should allow the next analyst to continue without repeating all work.

Include:

1. why the case matters;
2. current disposition and confidence;
3. affected entities;
4. key timeline events;
5. actions already taken;
6. evidence location;
7. unresolved questions;
8. recommended next steps;
9. deadlines or containment urgency.

Use Slack for approved collaboration and GitHub for official evidence, reports, feedback and submissions. Use WhatsApp only for urgent programme announcements or schedule changes, not as the authoritative case record.

8. Closure criteria

A case may be closed when:

- the alert has an evidence-supported disposition;
- required escalation or containment has been completed or accepted by the responsible operator;
- evidence and queries are documented;
- limitations are recorded;
- detection-quality feedback is submitted where needed;
- the final report is reviewed;

- credentials and sensitive data have been removed from the submission.

A lack of evidence is not automatically evidence of benign activity. Close as inconclusive or escalate when required information is unavailable.

9. Capstone scenario

The capstone combines authentication, endpoint, web and application telemetry from an authorised synthetic VCC scenario.

The learner must:

1. confirm the assigned pod scope shown by the toolkit;
2. capture the original Wazuh alert;
3. build a query journal;
4. correlate at least three telemetry sources;
5. construct a timeline;
6. identify affected entities;
7. state at least two competing explanations;
8. test those explanations against evidence;
9. assign disposition, severity and confidence;
10. recommend containment, escalation or tuning;
11. complete the incident report and evidence register;
12. submit through the approved GitHub assignment workflow.

10. Capstone assessment rubric

Area	Weight	Evidence of success
Scope and safety	10%	Uses only assigned synthetic evidence and protects credentials
Evidence collection	20%	Preserves relevant events and records sources accurately
Query method	15%	Queries are reproducible and logically sequenced
Correlation and timeline	20%	Links events across sources without unsupported claims
Disposition and confidence	15%	Conclusion is supported and limitations are explicit
Communication	10%	Report is clear, concise and suitable for handoff
Detection feedback	10%	Tuning or quality recommendation is safe and testable

11. Final analyst checklist

Before submission, confirm:

- every statement is labelled as fact, analysis, assumption or recommendation where needed;
- timestamps use a stated timezone;
- screenshots are readable and redacted;
- commands and queries can be reproduced;
- the assigned pod was not selected through a client-controlled parameter;
- no credential material is committed;
- limitations are honest;
- the report answers what happened, what was affected, how confident the analyst is and what should happen next.

Source: docs/wazuh-handbook/01-wazuh-architecture-and-neolabs-deployment.md

Wazuh SOC Level 1 Handbook — Module 1

Architecture, Data Flow and the NeoLabs Student Deployment

NeoLabs tested baseline: Wazuh 4.14.7
Module version: 1.0
Reconciled: 14 August 2026
Audience: Beginner-to-intermediate SOC Level 1 interns

> Wazuh is not one program running in one window. It is a set of components that collect, analyse, store and display security data. A SOC analyst should be able to identify which stage produced—or failed to produce—the evidence they are looking for.

For the current programme/runtime summary, also read `../../PROGRAMME_CURRENT_STATE.md`.

Learning objectives

By the end of this module, you should be able to:

- explain the roles of the Wazuh manager, Filebeat, indexer and dashboard;
- explain how the NeoLabs VCC telemetry feed differs from a normal Wazuh endpoint agent;
- describe how a VCC event becomes a searchable Wazuh alert;
- distinguish source `event_time`, replay/ingestion metadata and Wazuh index time;
- explain why an empty dashboard is not automatically evidence that no event occurred;
- use the current NeoLabs one-click Windows startup and Doctor workflow;
- explain the Night Watch and Telemetry Health views;
- identify the student/operator trust boundary and pod-isolation controls;
- perform a safe first-line data-flow check without exposing credentials.

---

1. What Wazuh is

Wazuh is an open-source security platform that can collect and analyse operating-system, application, network, cloud and security telemetry. In a typical deployment it combines endpoint agents/collectors, a Wazuh server or manager, Filebeat, a Wazuh indexer and a Wazuh dashboard.

For SOC work, think of the stack as a pipeline:

```
source event
→ collection
→ parsing/decoding
→ rule evaluation
→ alert creation
→ forwarding
→ indexing
→ search/dashboard
```

A failure at one stage can make later stages look empty even when earlier stages are healthy. That is why NeoLabs troubleshooting verifies the full chain rather than only checking that Docker containers are running.

---

2. Core Wazuh components

2.1 Wazuh agent

A Wazuh agent is software installed on a monitored endpoint. It can collect operating-system/application logs, Windows Event channels, file-integrity activity, system inventory and other endpoint security information.

NeoLabs VCC boundary

SOC interns do **not** install or administer Wazuh agents inside VCC application pods. The VCC remains operator-managed. The programme exports approved synthetic pod telemetry through a separate protected NeoLabs feed.

Therefore:

- absence of a pod-named Wazuh agent does not mean the VCC feed is absent;
- the student does not receive pod SSH/container access merely because Wazuh can see pod telemetry;
- student pod scope is controlled by the NeoLabs access service, not by a local Wazuh agent selector.

2.2 Wazuh manager

The Wazuh manager analyses received events. Important responsibilities include:

- reading configured log sources;
- decoding/parsing events;
- applying rules and correlations;
- writing alert records;
- coordinating supported Wazuh functions and API access.

The NeoLabs Compose service is:

```
wazuh.manager
```

The current manager configuration reads the shared NeoLabs VCC NDJSON stream at:

```
/var/ossec/logs/vcc/vcc-events.ndjson
```

with JSON log format.

2.3 Filebeat

Filebeat forwards Wazuh alerts from the manager toward the indexer.

This creates an important troubleshooting distinction:

```
manager parsed/matched the event
but
Filebeat/indexer did not make it searchable
```

That situation is different from a collector failure or a decoder/rule failure.

2.4 Wazuh indexer

The Wazuh indexer is the search/storage layer based on OpenSearch technology. It stores documents, mappings and searchable indices.

The NeoLabs Compose service is:

```
wazuh.indexer
```

Week 1 investigation primarily uses:

```
wazuh-alerts-*
```

Normal NeoLabs VCC baseline events are deliberately eligible to become searchable alerts so Operation Night Watch can show normal activity, not only high-severity detections.

2.5 Wazuh dashboard

The Wazuh dashboard provides the analyst interface for search, Threat Hunting/Discover, saved objects, dashboards and Wazuh modules.

The NeoLabs service is:

```
wazuh.dashboard
```

The normal student profile binds the dashboard to loopback:

```
https://127.0.0.1:8443
```

The dashboard, Wazuh API and indexer must not be exposed publicly merely to simplify student access.

---

3. The current NeoLabs student topology



Persistence

Docker named volumes retain manager/indexer/dashboard state across ordinary stops. A normal `git pull` or stack restart does not require students to delete Wazuh volumes or regenerate credentials.

A destructive reset is a separate local-only operation and is **not** normal troubleshooting.

---

4. How VCC telemetry reaches the student

The programme separates **telemetry access** from **infrastructure access**.

A SOC intern receives:

- the SOC toolkit;
- a private NeoLabs Access Code;
- the server-assigned pod/track/scenario context;
- a local Wazuh workstation;
- only the synthetic telemetry/evidence authorised for that assignment.

The intern does **not** receive:

- EC2 or pod shell access;
- database/container administration;
- broad AWS credentials;
- another pod's telemetry;
- mentor ground truth;
- a client-side control that authorises a different pod.

Server-authoritative scope

The student may type a pod number during login as confirmation, but the server-side assignment is authoritative. A local edit cannot turn a student into another pod/track.

Both live and replay paths validate the assigned scope. Replay additionally validates that records are synthetic, belong to the assigned pod and contain required event fields before they are appended to the local Wazuh telemetry stream.

---

5. LIVE versus REPLAY

The VCC uses a cost-aware hybrid runtime. The main VCC EC2 does not have to remain on for every hour of an investigation.

The same student access flow can represent different authorised SOC states:

- **LIVE** — protected live pod telemetry during approved interactive windows;
- **REPLAY** — archived telemetry for the assigned pod/scenario;
- **CLOUD_LIVE / ENDPOINT_LIVE** — scenario-specific cloud/endpoint evidence surfaces where applicable;
- **OFFLINE/no surface** — the toolkit must not pretend data is available.

Event time during replay

Replay preserves the original source:

```
event_time
```

and adds replay metadata separately.

For an incident timeline, order events by original event time unless the assignment specifically asks you to analyse collection/replay delay.

---

6. How Wazuh analyses a NeoLabs event

A simplified event path is:

```
NDJSON event
→ JSON decoding
→ NeoLabs base rule
→ child/correlation rules where applicable
→ alert record
→ Filebeat
→ Wazuh indexer
→ dashboard search
```

A synthetic VCC event may contain fields such as:

```
{
  "schema_version": "1.0",
  "event_id": "evt-auth-0009",
  "event_time": "2026-08-14T09:18:07Z",
  "pod_id": "pod-03",
  "event_type": "authentication",
  "action": "login",
  "outcome": "success",
  "user": "student.synthetic",
  "source_ip": "203.0.113.44",
  "session_id": "sess-902",
  "correlation_id": "corr-902",
  "synthetic": true
}
```

Wazuh may expose NeoLabs dynamic fields under `data.*`. In the current mapping, the NeoLabs user identity is commonly visible as `data.dstuser` in the saved Week 1 view.

---

7. NeoLabs custom rules

The current NeoLabs VCC rule set includes training rules for:

- normal VCC baseline events;
- authentication failures;
- repeated authentication failures;
- successful authentication;
- success after earlier failures;
- sensitive account changes;
- protected authorisation denials;
- telemetry collection/parser/visibility problems.

The base VCC event rule is intentionally searchable at a non-zero alert level so normal Week 1 activity can appear in `wazuh-alerts-*`.

Telemetry-health problems are represented around rule:

```
100150
```

A Wazuh rule match means the configured condition was observed. It does **not** automatically prove malicious intent or impact.

---

8. Current Windows setup and startup

The normal student workflow has changed from the older manual enrolment sequence.

First/current startup

From the latest toolkit checkout:

1. Start Docker Desktop with WSL2 integration.
2. Double-click:

```
START-NEOLABS-SOC.cmd
```

The launcher automatically:

1. verifies WSL2/toolkit prerequisites;
2. performs first-time Wazuh preparation only when needed;
3. preserves an existing `.env` and Wazuh data;

4. reuses a valid NeoLabs session or asks for assigned pod + private Access Code;
5. connects to the current authorised LIVE/REPLAY surface;
6. starts/waits for manager, indexer, dashboard and telemetry collector;
7. reloads the current NeoLabs rule file in the manager;
8. proves an assigned-pod VCC event is searchable in `wazuh-alerts-*`;
9. reports latest-event freshness;
10. applies/checks local alert-index retention/disk safety;
11. provisions Night Watch/Telemetry Health saved objects where supported;
12. copies the local Wazuh `admin` password to the Windows clipboard without printing it; and
13. opens the local dashboard.

Do not begin cohort analysis until the launcher displays:

```
SOC WORKSTATION READY
```

No global CLI/manual gateway requirement

For the normal Windows programme path, students do **not** need to:

- run `python -m pip install -e .`;
- add Python Scripts to Windows PATH;
- copy a private/manual gateway URL;
- manually enrol with a token file;
- manually select another telemetry target.

Advanced/manual scripts still exist for troubleshooting and non-Windows operation, but they are not the primary student onboarding sequence.

---

9. Wazuh dashboard login

After READY, the normal dashboard is:

```
https://127.0.0.1:8443
```

Use:

```
Username: admin
Password: the locally generated WAZUH_INDEXER_PASSWORD
```

On Windows, the launcher copies the password to the clipboard without displaying it. Press `Ctrl+V` at the login page, then replace the clipboard contents with non-sensitive text after login.

The following are **not** the human dashboard password:

- `WAZUH_API_PASSWORD` — internal Wazuh API/service communication;
- `WAZUH_DASHBOARD_PASSWORD` — internal dashboard service account.

Never paste the local Wazuh password into an assignment, screenshot, Slack post or Git commit.

---

10. Night Watch and Telemetry Health views

NeoLabs — Operation Night Watch

The toolkit attempts to provision a Week 1 pod-scoped saved view/dashboard containing important investigation fields such as:

- `data.pod_id`;
- `data.event_type`;
- synthetic user identity;
- `data.source_ip`;
- `data.outcome`;
- `data.correlation_id`;
- Wazuh `rule.id`, `rule.level`, `rule.description`;
- original `data.event_time`.

Use it to establish normal authentication/application/API behaviour and build reusable Week 1 searches.

NeoLabs — Telemetry Health

This view focuses on telemetry-quality problems, especially rule `100150`, so students can distinguish “no suspicious event” from “the collection/search pipeline is unhealthy.”

Saved-object provisioning is fail-soft. If an import is unavailable on a local dashboard instance, Threat Hunting/Discover remains the supported fallback.

---

11. Freshness and local retention

The current toolkit reports the newest assigned-pod event indexed in Wazuh.

Default freshness warning:

```
90 minutes
```

This is a health signal—not proof that an attack should happen every 90 minutes.

Current local alert-index defaults:

```
wazuh-alerts-* retention: 30 days
filesystem warning:       85%
filesystem critical:      92%
```

This retention applies only to the student's local Wazuh alert indices. It does **not** delete VCC server-side telemetry archives/evidence and does not force-overwrite unrelated existing index-management policies.

---

12. NeoLabs Doctor

On Windows, double-click:

```
CHECK-NEOLABS-SOC.cmd
```

or run:

```
.\neolabs.cmd doctor
```

Doctor checks the pipeline in order:

```
1. NeoLabs authentication
2. current LIVE/REPLAY telemetry surface
3. raw VCC event file
4. Wazuh rule engine
5. Filebeat
6. Wazuh indexer
7. Wazuh dashboard
```

It also reports telemetry freshness and local index/disk state.

This is the preferred first troubleshooting tool because it prevents the common mistake of changing Wazuh rules when the real failure is earlier in the pipeline.

---

13. Why an empty dashboard may be misleading

Before writing “no events were found,” confirm:

1. Doctor/indexer health passes;
2. the latest VCC event is reasonably fresh for the current assignment/window;
3. the filter uses the assigned `pod_id`;
4. the correct `wazuh-alerts-*` data view is selected;
5. the time range includes the original event times;
6. Telemetry Health does not show a collection/parser/visibility problem.

A zero-result query becomes useful evidence only after these assumptions are checked.

---

14. Alerts versus archives

`wazuh-alerts-*` contains searchable alert documents. The current NeoLabs Week 1 base rule ensures normal approved VCC baseline events can be represented there.

Wazuh archive indexing, when enabled, can provide additional events that did not become alerts. Archive indexing may not be available on every student workstation because it increases storage/resource/privacy requirements.

Do not assume `wazuh-archives-*` exists. If the assignment needs evidence that is not available in alerts, record the visibility limitation and use only mentor-approved evidence sources.

---

15. Network exposure and security boundaries

Common Wazuh ports can include agent communication/enrolment, server API, indexer and dashboard ports. In the NeoLabs student workstation:

- the dashboard is loopback-only by default;
- the Wazuh API/indexer are not intentionally published for student network access;
- internal components communicate on the Compose network;
- VCC telemetry scope is server-managed;
- students must not expose the local stack publicly for convenience.

The fact that Wazuh is a security tool does not make an insecure Wazuh deployment acceptable.

---

16. Safe manual checks

Advanced/manual checks remain available from the toolkit root, for example:

```
bash wazuh-stack/scripts/compatibility-check.sh
bash wazuh-stack/scripts/health-check.sh
bash wazuh-stack/scripts/verify-telemetry-pipeline.sh --wait 180
bash wazuh-stack/scripts/telemetry-freshness.sh
bash wazuh-stack/scripts/doctor.sh
```

Use `wazuh-logtest` through the supported verifier/Doctor workflow when checking decoding/rules. Do not broadly raise rule levels, disable TLS, publish indexer/API ports or delete volumes simply to make a dashboard show data.

---

17. Evidence and confidentiality

Useful Week 1 evidence includes:

- Wazuh event/rule IDs;
- assigned pod and explicit time range;
- relevant source event fields;
- saved/reproducible filters;
- original event-time timeline;

- telemetry-health/freshness notes;
- redacted screenshots.

Never submit:

- NeoLabs Access Code;
- Wazuh local password;
- session token;
- signed private URL;
- private key/certificate contents;
- AWS credentials;
- another pod's data;
- real customer/production information.

---

18. Week 1 practical checklist

For Operation Night Watch:

1. Start with `START-NEOLABS-SOC.cmd`.
2. Wait for `SOC WORKSTATION READY`.
3. Log in to the local Wazuh dashboard as `admin`.
4. Open NeoLabs — Operation Night Watch or Threat Hunting.
5. Confirm your assigned pod filter.
6. Record latest-event freshness and inspect Telemetry Health.
7. Find normal authentication and application/API activity.
8. Pivot using identity/source/session/correlation fields.
9. Save/reuse at least three baseline searches.
10. Build a short original-event-time timeline.
11. Record one visibility limitation.
12. Submit only redacted evidence through the central assignments repository.

---

19. Completion standard

You understand the NeoLabs Wazuh architecture when you can answer all of these without guessing:

- Where did this event originate?
- How did it reach the local telemetry file?
- Did Wazuh decode and match a rule?
- Did Filebeat/indexer make it searchable?

- Am I using the correct assigned pod and time range?
- Is the newest telemetry fresh/healthy?
- Is this rule an observation or proof of compromise?
- What can the current telemetry prove, and what can it not prove?

That pipeline awareness is one of the most important habits of a Level 1 SOC analyst.

Source: docs/dashboard-tutorials/01-orientation-and-alert-investigation.md

Wazuh Dashboard Tutorial 01 — Orientation and First Investigation

NeoLabs tested baseline: Wazuh 4.14.7
Reconciled: 2026-08-14
Audience: SOC Level 1 interns

Dashboard labels can vary by release. Current startup behaviour is documented in `../../PROGRAMME_CURRENT_STATE.md`.

Before opening the dashboard

Start through the platform root launcher:

```
Windows: START-NEOLABS-SOC.cmd
Linux:   ./start-neolabs-soc.sh
```

On a first Linux run, `bash start-neolabs-soc.sh` is also valid.

Wait for `SOC WORKSTATION READY`. This means the toolkit verified service health **and** that assigned-pod VCC telemetry is searchable in the local Wazuh indexer.

If anything looks wrong:

```
Windows: START-NEOLABS-SOC.cmd doctor
Linux:   ./start-neolabs-soc.sh doctor
```

Normal dashboard URL:

```
https://127.0.0.1:8443
```

Login username is `admin`. Windows copies the locally generated password to the clipboard without printing it. A headless Linux host prints secure port-forward guidance rather than exposing the dashboard publicly.

Current NeoLabs investigation views

NeoLabs — Operation Night Watch

Use this preconfigured pod-scoped saved view/dashboard for Week 1 when provisioning succeeds. It focuses on:

- assigned `pod_id`;
- `event_type`;
- NeoLabs user identity (`data.dstuser` in the current Wazuh mapping);
- `source_ip`;
- `outcome`;
- `correlation_id`;
- Wazuh rule ID/level/description;
- original source `event_time`.

NeoLabs — Telemetry Health

Use this saved view for collector/parser/visibility problems, especially NeoLabs rule `100150`.

Saved-object provisioning is fail-soft. If these views are unavailable, use **Threat Hunting/ Discover** against `wazuh-alerts-*`; do not reset Wazuh because a saved view did not import.

Alerts versus raw/archive data

Week 1 begins with `wazuh-alerts-*`. Normal VCC baseline events are deliberately eligible for searchable alerts so interns can see normal Night Watch activity, not only higher-severity detections.

`wazuh-archives-*` may not be enabled on every student workstation. If archive data is unavailable, state the visibility limitation rather than assuming the event never existed.

Time is evidence

For assignment evidence:

1. locate the relevant event(s);
2. choose an absolute UTC range around their **original event time**;
3. record that range in the query journal;
4. distinguish source `event_time` from replay/ingest/indexed timestamp.

Replay preserves original `event_time` and stores replay metadata separately.

First Week 1 investigation flow

1. Open **NeoLabs — Operation Night Watch** or Threat Hunting.
2. Confirm the assigned `pod_id`. If another pod appears, stop and contact a mentor.
3. Check latest-event freshness reported by the launcher/Doctor.
4. Inspect any Telemetry Health warning before interpreting missing data.
5. Set an absolute time range around Week 1 events.
6. Expand a record and capture `event_id`, original `event_time`, `event_type`, identity, source, outcome, session/correlation IDs and Wazuh rule metadata where available.
7. Find normal successful authentication and an ordinary failed authentication if present.
8. Pivot by identity → source IP → session/correlation ID to related application/API activity.
9. Identify storage/other approved telemetry visible to the pod.
10. Save/reuse at least three baseline filters/searches.
11. Build a short original-event-time timeline.
12. Record one visibility gap.

Week 1 is baseline building. “Unusual” does not automatically mean “malicious.”

Useful NeoLabs rule families

Current training rules include normal baseline events plus authentication failure/repeated failure, authentication success/success-after-failures, sensitive account change, authorisation denial and telemetry-health alerts.

Rule IDs/severity are pivots. Inspect underlying event fields/context before disposition.

Filters and pivots

A practical pivot sequence is:

```
pod
→ identity
→ source_ip
→ session_id / correlation_id
→ related application/auth/account activity
→ telemetry health for the same window
```

Do not reproduce denied/cross-pod requests merely to create evidence.

WQL/search-bar caution

Wazuh Query Language, OpenSearch/Discover query syntax and UI filter pills are not interchangeable everywhere. Prefer UI field filters when following this tutorial. If using a query language, record which view/search bar accepted it so another analyst can reproduce the result.

Zero results are not automatically evidence of absence

Before writing “no events found”:

- confirm Doctor/indexer PASS;
- confirm latest VCC event freshness;
- confirm the correct data view/index pattern;
- confirm the assigned pod filter;
- confirm absolute time range/time zone;
- inspect Telemetry Health / rule `100150`;
- state missing archive/source coverage if relevant.

Evidence capture

A useful screenshot should show enough context to prove the claim: view/search name, time range, active filters and relevant event/rule fields. Redact the local Wazuh password, NeoLabs Access Code/session, signed URLs, private keys/certificates and unrelated personal data.

Dashboard/agent distinction

The NeoLabs VCC pod feed is consumed as a protected telemetry stream/local file path; it is not permission to install/manage an agent inside a VCC pod. Absence of a pod-named Wazuh agent does not mean the VCC feed is absent.

When to use Doctor instead of changing settings

Use the platform launcher `doctor` action before editing dashboard/indexer/manager settings. Doctor identifies whether the break is at authentication, replay/live access, raw event, rule engine, Filebeat, indexer or dashboard.

Do not weaken TLS, expose indexer/API ports, disable rules broadly or delete local volumes simply to make the dashboard show data.

Completion check

You can complete this tutorial when you can explain the assigned pod, current data source/time range, one normal authentication sequence, one application/API pivot, the meaning of the relevant Wazuh rule, one saved/reproducible filter and one visibility limitation—without confusing replay/index time with source event time.

Source: docs/setup/WORKSTATION_COMPATIBILITY.md

NeoLabs SOC L1 Workstation Compatibility

For current startup behaviour see `../../PROGRAMME_CURRENT_STATE.md`.

Supported baseline

Requirement	Hard floor	Preferred minimum	Recommended
CPU architecture	64-bit x86_64 or arm64	64-bit x86_64	x86_64
CPU capacity	4 logical cores	4	6+
Memory visible to Linux/WSL2	7 GiB	8 GiB	12–16 GiB
Free disk	25 GiB	25 GiB	50 GiB
Container runtime	Docker Engine/Desktop	Docker Engine/Desktop	current supported release
Compose	v2	v2	current supported v2
`vm.max_map_count`	262144	262144	>=262144

Seven GiB is the constrained Week 1 memory floor. The launchers can install software and configure a kernel setting; they cannot fix insufficient RAM, CPU, disk capacity, blocked virtualisation or organisation policy.

Physical Windows 10/11 + WSL2

Use only:

```
START-NEOLABS-SOC.cmd
```

The Windows path is supported for **physical Windows 10/11 workstations**. Before attempting WSL/Docker setup, the internal bootstrap checks Windows OS type and common VM/VPS hardware identifiers. It fails early when it detects Windows Server or a Windows VM/VPS guest and directs the intern to an Ubuntu/Debian VPS instead.

On a supported Windows workstation the launcher owns:

- WSL2 detection/bootstrap;
- an existing current WSL2 distribution, or Ubuntu installation only when no Linux distro exists;

- Docker Desktop detection/installation/startup;
- Linux container engine + WSL integration verification;
- missing Linux packages inside WSL;
- `vm.max_map_count` adjustment through the WSL root account;
- Wazuh first-run preparation and subsequent startup.

Ubuntu is not mandatory when the intern already has a suitable Kali, Debian, Ubuntu or other current WSL2 distro. Git Bash/MSYS/Cygwin is not a supported Wazuh runtime.

Windows can require one restart after enabling WSL or one initial launch of a newly installed Linux distro to create its Linux user. Rerun the same root CMD afterward.

If WSL2 cannot start on a **physical** Windows computer because firmware virtualisation is disabled, the intern must enable Intel VT-x/AMD-V/virtualisation in the computer BIOS/UEFI. The launcher cannot change a physical machine's BIOS/UEFI setting itself.

Ubuntu / Debian Linux

Use:

```
bash start-neolabs-soc.sh
```

Run it as the normal Linux account, **not** with `sudo` in front of the whole command. The launcher invokes administrator privileges itself only for:

- package/repository installation;
- Docker service/group setup;
- `vm.max_map_count` configuration/persistence.

When Docker is absent on Ubuntu/Debian, the launcher installs Docker Engine + Compose v2 and then gives the normal user Docker access. Wazuh runtime commands themselves should then run without `sudo`.

VPS / remote server policy

For an intern who chooses a VPS or remote server, the supported platform is **Ubuntu or Debian Linux only**.

Recommended images:

- Ubuntu 24.04 LTS;
- Ubuntu 22.04 LTS;
- a current Debian release.

Do not provision Windows Server or a Windows desktop VM/VPS for the NeoLabs SOC workstation. WSL2 inside a Windows guest depends on nested virtualisation being exposed by the host hypervisor/provider, which cannot be enabled by the NeoLabs script from inside the guest. The programme avoids that dependency by running Docker Engine/Wazuh directly on Linux VPS hosts.

On a VPS:

```
bash start-neolabs-soc.sh
```

The Wazuh dashboard remains loopback-only. The launcher prints an SSH port-forward example for headless remote access.

Other Linux distributions

The root Bash launcher can operate when its prerequisites already exist. Automatic Docker installation is intentionally bounded to Ubuntu/Debian. On another distribution, install Docker Engine + Compose v2 through that distribution's supported method, then rerun the same launcher. For programme-provisioned VPS systems, use Ubuntu/Debian rather than another distribution.

Headless Linux server

Wazuh still binds the dashboard to loopback. If the server has no desktop/browser, the launcher prints a secure SSH local-port-forward example. Do not change the dashboard binding to `0.0.0.0` merely to make remote access easier.

Unsupported configurations

- Windows Server as the SOC workstation;
- Windows VM/VPS guests as the SOC workstation;
- VPS/remote SOC workstations using a non-approved OS instead of Ubuntu/Debian;
- 32-bit operating systems;
- Docker Compose v1/legacy Docker Toolbox;
- less than 7 GiB Linux/WSL2-visible memory;
- systems where local Docker cannot be used;
- Git Bash/MSYS/Cygwin as the Wazuh runtime;
- public exposure of the local Wazuh administrative services;
- production/company shared infrastructure used as an internship target/workstation without explicit approval.

Diagnostics

Windows physical workstation:

```
START-NEOLABS-SOC.cmd doctor
```

Linux / Ubuntu / VPS:

```
./start-neolabs-soc.sh doctor
```

Mentors/advanced troubleshooting may still run `bash wazuh-stack/scripts/compatibility-check.sh`, but that internal check is diagnostic only. Students should not repair its failures by guessing where to add `sudo`; the platform launcher owns supported setup changes.

Source: docs/setup/BACKUP_AND_RECOVERY.md

NeoLabs SOC L1 Backup and Recovery

Purpose

The student Wazuh environment contains locally generated investigation data, dashboards, indexer data and manager state. A workstation failure should not require a learner to rebuild every investigation from the beginning.

The toolkit therefore provides a guarded Docker-volume backup and restore process. It is designed for local training data, not for production disaster recovery.

Security boundary

A toolkit backup deliberately excludes:

- `.env`;
- VCC bootstrap tokens;
- client private keys;
- client certificates and CA trust files;
- enrolment responses;
- private VCC endpoints;
- production data or credentials.

After restoring, the learner must complete a fresh operator-approved enrolment. This prevents copied backups from becoming reusable VCC access packages.

Create a backup

From the repository root:

```
bash wazuh-stack/scripts/backup.sh
```

The script performs the following actions:

1. confirms that Docker, Compose and the local `.env` file are available;
2. discovers the named volumes declared by the Wazuh Compose file;
3. stops the running stack to obtain a consistent point-in-time snapshot;
4. creates one compressed archive per existing volume;
5. calculates SHA-256 checksums;
6. writes a manifest and recovery notice;
7. restarts the stack unless `--no-restart` was supplied.

Use a specific destination when required:

```
bash wazuh-stack/scripts/backup.sh --output /secure/local/path/neolabs-backup
```

Keep the backup on encrypted, personally controlled storage. Do not upload it to a public repository or share it in Slack or WhatsApp.

Verify a backup

Always verify a backup before relying on it:

```
bash wazuh-stack/scripts/verify-backup.sh /path/to/backup
```

Verification checks the manifest version, checksum file, archive readability, presence of at least one volume and absence of common credential filenames.

Restore a backup

A restore replaces the contents of the matching Wazuh volumes. Stop the stack and confirm the backup path before continuing:

```
bash wazuh-stack/scripts/stop.sh
bash wazuh-stack/scripts/restore.sh /path/to/backup --force
```

The restore script:

- verifies checksums before modifying volumes;
- accepts only volume names declared in the current Compose file;
- refuses to run while stack services are active;
- recreates and repopulates the declared volumes;
- removes stale local VCC collector scope and cursor state;
- does not restore enrolment credentials.

After restoration:

```
bash wazuh-stack/scripts/compatibility-check.sh
bash wazuh-stack/scripts/start.sh
bash wazuh-stack/scripts/health-check.sh
```

Then request a new enrolment token from the programme operator and complete the normal enrolment workflow.

Rehearsal

The repository CI performs a synthetic Docker-volume recovery rehearsal using:

```
bash wazuh-stack/tests/backup-restore-rehearsal.sh
```

The rehearsal creates a temporary volume, writes a synthetic marker, archives it, verifies its checksum, deletes and recreates the volume, restores the archive and confirms that the marker survived. It never reads the learner's Wazuh data.

Recovery limitations

A successful archive verification does not prove that every future Wazuh release can read data created by an older release. Restore into the same pinned toolkit version first. Complete a documented Wazuh upgrade only after the restored stack is healthy.

Backups are not a substitute for GitHub submissions. Investigation reports, evidence logs and query journals should still be committed to the approved assignment repository after removing credentials, private URLs and sensitive evidence.

Source: troubleshooting/WAZUH_SETUP_AND_TROUBLESHOOTING_GUIDE.md

NeoLabs Wazuh Setup and Troubleshooting Guide

Tested baseline: Wazuh 4.14.7
Reconciled: 2026-08-14
Scope: learner-owned workstation + authorised synthetic VCC telemetry only

1. Start with the platform root launcher

Windows:

```
START-NEOLABS-SOC.cmd doctor
```

Linux/Ubuntu:

```
./start-neolabs-soc.sh doctor
```

Do not start troubleshooting by running random files in `wazuh-stack/scripts/` or by adding `sudo` to a command that failed. The root launchers own supported prerequisite installation and privilege changes; the lower-level Wazuh scripts are intentionally runtime/validation components.

2. Golden path

```
NeoLabs authentication / server assignment
→ current LIVE/REPLAY surface
→ raw assigned-pod VCC event file
→ Wazuh manager localfile input
→ NeoLabs Wazuh rules
→ Filebeat
→ Wazuh indexer (wazuh-alerts-*)
→ dashboard / saved views / time filters
```

`SOC WORKSTATION READY` is displayed only after assigned-pod telemetry is actually searchable in the indexer.

3. First-run prerequisite problems

Windows

Rerun `START-NEOLABS-SOC.cmd` after resolving the exact state it reports. The launcher handles supported WSL2/Docker Desktop/bootstrap work. Legitimate one-time interruptions include a Windows restart after enabling WSL, launching a newly installed Linux distro once to create its Linux user, or resolving a Docker Desktop first-run/virtualisation/update prompt.

Do not run the Wazuh stack from Git Bash/MSYS/Cygwin.

Linux / Ubuntu

Run the launcher as the normal user:

```
bash start-neolabs-soc.sh
```

Do **not** run `sudo ./start-neolabs-soc.sh`. The launcher invokes `sudo` itself only for package installation, Docker service/group setup and kernel configuration.

On Ubuntu/Debian the launcher can install Docker Engine + Compose v2 when missing. If it adds the user to the Docker group, it attempts to refresh that group context immediately; where the shell cannot do so, log out/in once and rerun the same root launcher.

4. Docker problems

Useful mentor/advanced checks:

```
docker --version
docker compose version
docker info
```

On Windows, Docker must be the Linux engine and reachable from the same WSL2 distro used by the toolkit. On native Linux, the normal user must be able to run `docker info` without prefixing every Wazuh command with `sudo`.

Never expose an unauthenticated Docker TCP socket to solve a permissions problem.

5. Memory, disk and indexer kernel requirement

Current workstation baseline:

- 7 GiB Linux/WSL-visible RAM hard floor;
- 8 GiB preferred;

- 12–16 GiB recommended;
- 25 GiB free disk hard floor; 50 GiB recommended;
- `vm.max_map_count >= 262144`.

The root launcher configures `vm.max_map_count` when it can. If an organisation/server policy blocks that administrator change, the host administrator must resolve the policy; do not edit Wazuh rules/Compose to bypass the indexer's requirement.

Local indexer filesystem warnings begin at 85% and become critical at 92%. Preserve required evidence before any mentor-approved cleanup; do not delete Wazuh volumes merely to clear a warning.

6. Local Wazuh configuration

The launchers generate `wazuh-stack/.env` only when it is absent and preserve an existing file. Do not regenerate local secrets because the repository was updated.

Human dashboard login:

```
Username: admin
Password source: local WAZUH_INDEXER_PASSWORD
```

`WAZUH_API_PASSWORD` and `WAZUH_DASHBOARD_PASSWORD` are internal service credentials, not the human login.

7. NeoLabs authentication/runtime failures

Use the root launcher actions:

```
Windows: START-NEOLABS-SOC.cmd status
         START-NEOLABS-SOC.cmd login

Linux:   ./start-neolabs-soc.sh status
         ./start-neolabs-soc.sh login
```

The pod entered during login is confirmation only; server assignment remains authoritative. Do not edit runtime state or a base URL to reach another pod.

8. Replay/live telemetry not arriving

Run Doctor, then verify the server state is an authorised SOC surface, raw telemetry contains synthetic assigned-pod records, replay validation did not reject pod/scenario fields, live enrolment is current when LIVE, and internet/TLS connectivity is available.

Replay preserves original `event_time`; replay time is not the incident sequence.

9. Raw events exist but Wazuh has no alerts

The manager must monitor `/var/ossec/logs/vcc/vcc-events.ndjson` as JSON and load `config/rules/neolabs_vcc_rules.xml`.

Normal startup restarts the manager after asserting the stack so current rules are loaded after `git pull`. The telemetry verifier uses `wazuh-logtest` as part of the rule-engine proof.

Do not broadly raise rule levels simply to make a dashboard busy.

10. Filebeat/indexer problems

If raw events/rule matching works but `wazuh-alerts-*` remains empty, Doctor separates Filebeat and indexer checks. Verify indexer health before changing mappings/policies. NeoLabs retention is deliberately scoped to local `wazuh-alerts-*` and does not force-overwrite unrelated policies.

11. Dashboard opens but looks empty

Before concluding “no events”:

1. confirm Doctor/indexer PASS;
2. check latest indexed VCC event freshness;
3. confirm assigned `pod_id` filter;
4. use `wazuh-alerts-*`, Threat Hunting or **NeoLabs — Operation Night Watch**;
5. set a correct absolute UTC range around original event times;
6. inspect **NeoLabs — Telemetry Health** / rule `100150`.

A zero-result query is useful evidence only after source health and time/filter assumptions are verified.

12. Saved Night Watch/Telemetry Health views missing

Saved-object provisioning is fail-soft. Healthy telemetry/indexing does not depend on the saved view import. Continue through Threat Hunting and report the view/import problem; do not reset Wazuh data merely because a saved object is absent.

13. Safe bounded repair

The root startup path permits one bounded local telemetry repair. It may reassert local services and, in replay mode, refetch the authorised replay state. It does not reset VCC server state, switch pods, delete Wazuh volumes or fetch another student's data.

If searchability still cannot be proven, stop relying on that workstation for assignment conclusions and send redacted Doctor output to the mentor.

14. Headless Linux dashboard access

The dashboard remains loopback-only. A headless server launcher prints an SSH local-forward example. Use that secure tunnel from the intern's own computer instead of changing the dashboard binding to a public address.

15. Advanced internal commands

Mentors may inspect `wazuh-stack/scripts/` directly for targeted diagnosis, backup/restore or CI rehearsal. These are not student setup steps.
`reset.sh --confirm-destroy-local-data` is destructive local recovery and is never a normal startup fix.

16. Evidence to send a mentor

Useful redacted evidence includes Doctor PASS/WARN/FAIL lines, current assigned pod/scenario/runtime state without credentials, latest event freshness, `docker compose ps` status, small relevant logs and event/rule IDs/screenshots.

Never send `.env`, admin password, Access Code, session token, private key/certificate contents, signed private URLs or another pod's data.

17. Stop conditions

Stop/escalate on cross-pod events/access, real personal/production data, exposed credentials/private keys, unexpected infrastructure access, external-target traffic or service instability. Do not weaken isolation/security controls to make a troubleshooting check pass.

Source: wazuh-stack/README.md

NeoLabs Student Wazuh Stack

Purpose

This directory contains the local Wazuh 4.14.7 manager, indexer, dashboard, telemetry collector, rules and maintenance scripts used by authorised NeoLabs SOC Level 1 interns.

This directory is an implementation/runtime directory, not the normal student start location.

Student entry points

From the repository root:

Windows

```
START-NEOLABS-SOC.cmd
```

Linux / Ubuntu

```
bash start-neolabs-soc.sh
```

The platform launcher owns prerequisite installation, kernel configuration, first-run secret/config generation, Wazuh startup, NeoLabs authentication, telemetry connection, index verification and dashboard access. Students should not assemble a startup process by directly invoking scripts in this directory.

Diagnostics are also routed through the root launcher:

```
Windows: START-NEOLABS-SOC.cmd doctor
Linux:   ./start-neolabs-soc.sh doctor
```

Internal telemetry path

```
VCC live/archive
→ authorised NeoLabs LIVE/REPLAY access
→ local vcc-events.ndjson
→ Wazuh manager JSON localfile input
→ NeoLabs VCC rules
→ Filebeat
→ Wazuh indexer wazuh-alerts-*
→ dashboard / Threat Hunting / saved views
```

Replay validates `synthetic=true`, assigned `pod_id`, required fields and scenario scope before append. Original `event_time` is preserved and replay metadata remains separate.

READY contract

`SOC WORKSTATION READY` means a real synthetic event for the server-assigned pod has been processed and is searchable through the local Wazuh indexer. Container health alone is insufficient. If searchability initially fails, the root launcher permits only one bounded local repair attempt before refusing READY.

Dashboard

Normal local URL:

```
https://127.0.0.1:8443
```

Human username is `admin`. The human password is the locally generated `WAZUH_INDEXER_PASSWORD` in private `.env`. Internal `wazuh-wui` / `kibanaserver` credentials are separate service credentials.

Current NeoLabs rules

`config/rules/neolabs_vcc_rules.xml` includes training rules for normal VCC baseline events, authentication failures/repetition, successful authentication, success after failures, sensitive account changes, protected authorisation denials and telemetry-health problems (`100150`).

The base NeoLabs VCC rule produces searchable baseline documents, which is required for Operation Night Watch. Rule matches are investigation pivots, not automatic proof of malicious activity.

`start.sh` explicitly restarts the Wazuh manager after Compose assertion so updated bind-mounted NeoLabs rules are loaded after a toolkit update.

Saved investigation views

Runtime provisioning attempts to create:

- **NeoLabs — Operation Night Watch** — assigned-pod baseline investigation view;
- **NeoLabs — Telemetry Health** — rule `100150` collection/parser/visibility troubleshooting view.

Saved-object provisioning is fail-soft; Threat Hunting remains available if local saved-object import fails.

Freshness and local retention

- telemetry freshness warning default: **90 minutes**;
- local `wazuh-alerts-*` retention default: **30 days**;
- filesystem warning: **85%**;
- critical: **92%**.

These controls affect the student's local Wazuh data only; they do not delete VCC server-side telemetry/evidence.

Workstation baseline

Requirement	Hard floor	Preferred	Recommended
Linux/WSL2-visible memory	7 GiB	8 GiB	12–16 GiB
CPU	4 logical cores	4	6+
Free disk	25 GiB	25 GiB	50 GiB
Compose	v2	v2	current v2
`vm.max_map_count`	262144	262144	>=262144

The root launchers now own the supported privilege changes required to reach these software/kernel prerequisites. `compatibility-check.sh` and `preflight.sh` deliberately remain read-only validation scripts and should not contain scattered `sudo` commands.

Internal/mentor operations

The scripts below remain available for CI, recovery and advanced mentor troubleshooting, but are not student-facing setup choices:

```
scripts/compatibility-check.sh
scripts/generate-local-secrets.sh
scripts/prepare-stack.sh
scripts/preflight.sh
scripts/start.sh
scripts/health-check.sh
scripts/verify-telemetry-pipeline.sh
scripts/repair-telemetry-pipeline.sh
scripts/telemetry-freshness.sh
scripts/doctor.sh
scripts/backup.sh / verify-backup.sh / restore.sh
scripts/stop.sh / reset.sh
```

Do not add `sudo` to those runtime commands simply because Docker/kernel setup failed. Correct the platform setup through the root launcher. `reset.sh` is destructive local recovery and is not normal Week 1 troubleshooting.

Private files

Never commit/share `.env`, NeoLabs Access Codes/session files, VCC private keys/certificates/private signed URLs, local Wazuh passwords, enrolment/runtime state, generated backups or another pod's evidence.

Network/security design

- Dashboard binds to loopback.
- Indexer and Wazuh API are not published for student network access.
- Internal components remain on the Compose internal network.
- Pod scope is server-managed.
- Collector/replay validation rejects wrong-pod/non-synthetic data.

Full troubleshooting guide:
`../troubleshooting/WAZUH_SETUP_AND_TROUBLESHOOTING_GUIDE.md`.

Source: templates/evidence-log-template.md

NeoLabs SOC Evidence Log Template

> Record evidence used in an authorised investigation. This template is for training and case discipline; it does not replace organisational forensic procedures or legal chain-of-custody requirements.

Case information

Field	Entry
Case / assignment ID	
Analyst	
Assigned pod	
Investigation start	YYYY-MM-DD HH:MM UTC
Report path / link	

Evidence register

Evidence ID	Date/ time collected (UTC)	Event time range	Source system	Evidence type	Collection method / query	Original location	File or record hash, when applicable	Handling / redaction note	Releva
EV-001				Alert / raw event / export / screenshot / config / owner confirmation					

Evidence-quality checks

For each important item, consider:

- Was the source operating during the event period?
- Is the timestamp event time, ingest time or alert time?

- Is the time zone known?
- Could the record be duplicated, delayed or truncated?
- Did a decoder or parser alter the displayed fields?
- Is the value source-generated, user-controlled or enrichment data?
- Is the evidence complete enough for the conclusion?
- Was sensitive data redacted without removing necessary meaning?

Evidence statements

Use one statement per important claim.

EV-___

Observed fact:

Source and time:

What this evidence supports:

What this evidence does not prove:

Quality or visibility limitation:

Related evidence:

Transfer or review record

Date/time UTC	Evidence ID(s)	From	To / reviewer	Purpose	Method	Confirmation

Final checks

- [ ] Every report fact points to an evidence ID or clearly identified source.
- [ ] No credential, private key or enrolment token is included.
- [ ] Personal or cross-pod information is removed or escalated rather than published.
- [ ] Screenshots show the required context and do not replace available raw records.
- [ ] UTC conversions are documented.
- [ ] Negative findings include the source, query and time range.

Source: templates/query-journal-template.md

NeoLabs SOC Query Journal Template

> Record searches as they are performed so another analyst can reproduce the investigation. Replace secrets and private endpoints with approved references; never paste enrolment tokens or private keys.

Case information

Field	Entry
Case / assignment ID	
Analyst	
Assigned pod	
Investigation date	
Primary alert	

Query register

Query ID	Time run (UTC)	Platform / source	Exact query or filter	Event-time range	Result count	Important result or negative finding	Evidence IDs	Next pivot
Q-001		Wazuh / OpenSearch / local tool						

Detailed query record

Q-___

Question being tested:

Data source and index/view:

Exact query/filter/command:

Time field used: Event time / ingest time / other
Time range:
Fields displayed or exported:
Result summary:
Relevant evidence IDs:
Limitations:
Decision / next query:

Query-quality checks

- [ ] The query uses the intended field rather than only free-text search.
- [ ] Exact and analysed fields are distinguished where applicable.
- [ ] The event-time field and time zone are recorded.
- [ ] A zero-result search was checked against source health and retention.
- [ ] Filters inherited from a dashboard or saved view are recorded.
- [ ] Wildcards and broad searches are narrowed before drawing conclusions.
- [ ] Commands were run only against synthetic data or explicitly authorised systems.
- [ ] Output containing sensitive values was redacted before submission.

Investigation summary

Most useful query:

Most important negative finding:

Visibility gap discovered:

Queries recommended for higher-tier review:

Source: templates/incident-report-template.md

NeoLabs SOC Incident Report Template

> Use for authorised VCC exercises and approved synthetic investigations. Replace instructional text before submission. Do not include credentials, private keys, enrolment tokens or unredacted personal information.

1. Report control

Field	Entry
Report title	
Case / assignment ID	
Analyst name	
Track and cohort	SOC Level 1 /
Assigned pod	
Report version	1.0
Prepared at	YYYY-MM-DD HH:MM UTC
Reviewer	
Classification	Training use / confidential programme material as directed

2. Executive summary

Write 3–6 sentences for a reader who may not know Wazuh or the scenario. State:

- what triggered the investigation;
- the affected account, service or asset;
- the most important confirmed facts;
- the current classification and confidence;
- the recommended next action.

Do not include unsupported claims or long raw-log extracts.

3. Alert and detection information

Field	Entry
Alert name	
Alert / rule ID	
Detection source	
Alert creation time	
Earliest confirmed event time	
Tool-assigned severity	
Analyst-assigned priority	
Relevant ATT&CK behaviour	Optional; explain mapping

4. Scope

Affected identities

Identity	Type	Role / context	Confirmed or suspected impact

Affected assets and services

Asset / service	Identifier	Criticality / role	Evidence of impact

Network, file and cloud indicators

Indicator	Type	Source	Relevance and limitation

5. Observed facts

List facts directly supported by evidence. Number each fact so it can be referenced later.

1.
2.
3.

6. Timeline

Use UTC unless the assignment specifies otherwise. Separate event time from ingest or alert time when relevant.

UTC time	Source	Entity	Observed activity	Evidence reference	Confidence

7. Evidence reviewed

Evidence ID	Source	Collection / query method	Time range	Integrity or quality note	Location / reference
EV-001					

Use the separate evidence log for detailed records.

8. Queries performed

Query ID	Data source	Query / filter	Time range	Result summary	Next pivot
Q-001					

Use the query journal when more detail is required.

9. Analysis

Interpretation

Explain what the combined facts suggest. Connect evidence through identity, time, process, network, session, request, file or resource pivots.

Alternative explanations considered

Alternative	Evidence supporting it	Evidence against it	Status
			Open / unlikely / ruled out

Visibility gaps and limitations

State missing data, parser failures, disconnected sources, clock issues, retention gaps or permissions that limit the conclusion.

10. Classification and confidence

Field	Entry
Classification	False positive / benign positive / suspicious / confirmed incident / insufficient evidence / data-quality issue
Confidence	Low / medium / high
Confidence reasoning	
Potential severity	Informational / low / medium / high / critical, according to assignment definitions
Business or lab impact	

11. Actions and recommendations

Actions already taken

Record only authorised actions actually performed.

1.
2.

Recommended next actions

Priority	Recommendation	Owner / escalation target	Reason	Deadline

Do not claim that a recommendation has been implemented unless evidence confirms it.

12. Escalation decision

Field	Entry
Escalated?	Yes / no
Escalated to	
Time	
Reason	
Information supplied	
Questions requiring higher-tier review	

13. Lessons and detection improvements

- Which field or source was most valuable?
- Which evidence was missing?
- Did the detection title and severity accurately describe the activity?
- What safe tuning, logging or playbook improvement is recommended?
- What should the analyst do differently next time?

14. References and attachments

List approved screenshots, exports, evidence IDs, task instructions and public references. Use repository-relative paths where possible. Redact sensitive values.

15. Reviewer notes

Reviewer	Date	Decision	Required corrections
		Approved / revise	

Source: labs/01-authentication-triage/README.md

Practice Lab 01 — Authentication Failure Chain

Track: SOC Level 1
Difficulty: Beginner → intermediate
Estimated time: 60–90 minutes
Dataset: `sample-logs/authentication/failed-login-chain.ndjson`
Scope: Synthetic `pod-03` training records only

Scenario

A Wazuh alert reports repeated authentication failures involving a service account. A successful login appears later in the same period, but one telemetry source also reports a collection gap.

Your task is to perform a defensible Level 1 triage. Do not assume that the alert proves compromise, and do not ignore the visibility gap.

Learning objectives

You should be able to:

- distinguish normal and suspicious authentication sequences;
- pivot by account, source address, session and correlation identifiers;
- build an event-time timeline;
- identify what the available records prove and what remains uncertain;
- recognise a cross-pod access-control denial without attempting to reproduce it;
- document a telemetry gap and explain its effect on confidence;
- prepare an incident report, evidence log and escalation package.

Safety boundary

- Use only the supplied synthetic dataset or the approved local Wazuh stack.
- Do not send requests to any VCC pod, endpoint or public address as part of this practice lab.
- Reserved example IP addresses in the dataset are documentation values, not targets.
- Do not attempt to change pod scope or access another pod.

Preparation

Study:

1. `docs/secops-foundations/01-security-operations-foundations.md`
2. `docs/secops-foundations/02-alert-triage-evidence-and-escalation.md`
3. `docs/secops-foundations/03-siem-pipelines-and-log-quality.md`
4. The Log Literacy manual sections on time, correlation and evidence statements

Copy these templates into your working folder:

- `templates/incident-report-template.md`
- `templates/evidence-log-template.md`
- `templates/query-journal-template.md`

Part A — Validate the dataset

Before analysing the security activity:

1. Confirm that every non-empty line is valid JSON.
2. Count the records.
3. Identify the schema version and pod identifier.
4. Confirm that each event is marked `synthetic: true`.
5. Compare `event_time` with `ingest_time` for delayed records.
6. Note any event type that represents telemetry health rather than user activity.

Example local validation command:

```
python3 - <<'PY'
import json
from pathlib import Path

path = Path("sample-logs/authentication/failed-login-chain.ndjson")
records = [json.loads(line) for line in path.read_text().splitlines() if line.strip()]
print(f"records={len(records)}")
print(f"pods={sorted({record['pod_id'] for record in records})}")
print(f"schemas={sorted({record['schema_version'] for record in records})}")
print(f"all_synthetic={all(record.get('synthetic') is True for record in records)}")
PY
```

Record this command and result in your query journal.

Part B — Authentication pivots

Answer the following with evidence IDs and exact queries:

1. Which accounts experienced authentication failures?
2. Which source addresses generated failures?
3. How many failures involved `svc-backup`?

4. Did any successful authentication follow those failures?
5. Which session was created by the relevant success?
6. Did the same source target additional accounts?
7. Which later events reference the suspicious session?
8. Is there a normal failure-followed-by-success sequence elsewhere in the dataset? How does its context differ?

You may use Python, `jq`, Wazuh dashboard filters or another approved local tool. Record the exact method.

Part C — Timeline

Build a timeline beginning five minutes before the first relevant failure and ending five minutes after the session revocation.

Your timeline should include:

- authentication failures;
- successful authentication;
- application activity;
- authorization decisions;
- account changes;
- session action;
- telemetry-health events.

Use **event time** for the primary order and note meaningful ingest delay separately.

Part D — Evidence reasoning

Write an evidence statement for each of the following:

1. Repeated failures for the service account.
2. Successful login from the same source.
3. Activity performed through the new session.
4. The denied cross-pod request.
5. The sensitive account change.
6. The process-telemetry gap.

For each statement, include:

- what the record directly proves;
- what it suggests in context;
- what it does not prove;
- the next source that would reduce uncertainty.

Part E — Classification

Choose one current classification:

- false positive;
- benign positive;
- suspicious;
- confirmed synthetic incident;
- insufficient evidence;
- data-quality issue.

Then provide:

- confidence: low, medium or high;
- potential severity;
- the strongest supporting facts;
- alternative explanations considered;
- the effect of the telemetry gap;
- whether the case should be escalated.

The quality of your reasoning matters more than selecting a label without explanation.

Part F — Deliverables

Submit the following through the programme's approved assignment repository when this lab is formally assigned:

```
lab-01-authentication-triage/
├── incident-report.md
├── evidence-log.md
├── query-journal.md
└── screenshots/
    └── README.md
```

The screenshots folder may contain approved Wazuh views, but do not include credentials, private URLs, unrelated alerts or another pod's data.

Review checklist

- [ ] Facts are separated from interpretation.
- [ ] Every important claim cites an evidence ID.
- [ ] Queries include their time ranges and data source.
- [ ] A zero-result search is not treated as proof without checking source health.
- [ ] The cross-pod denial is documented, not reproduced.
- [ ] The telemetry gap is included in the confidence assessment.
- [ ] No secret or private endpoint appears in the submission.
- [ ] The final recommendation stays within SOC Level 1 authority.

Instructor separation

This public repository intentionally contains no hidden answer key or scenario ground truth. Mentor review material belongs in a separate private repository.

Source: references/query-command-reference/README.md

NeoLabs SOC Level 1 Query and Command Reference

Version: 0.2-draft
Review baseline: 1 August 2026
Test scope: Synthetic files, the learner's local Wazuh stack and explicitly authorised VCC data only

> A query is a question expressed in a platform's syntax. Record the exact query, data source, time field and time range so another analyst can reproduce the result.

1. Choose the correct language

Interface or source	Primary syntax	Typical use
Wazuh specialised tabs and server API	Wazuh Query Language (WQL)	Filter agents, rules, decoders, groups and supported API resources
Wazuh/OpenSearch Discover	Dashboard search syntax and field filters	Search indexed alert or archive documents
Wazuh Indexer Dev Tools	OpenSearch Query DSL	Structured Boolean, range, aggregation and exact-field queries
NDJSON/JSON files	`jq` or Python	Validate and analyse synthetic datasets
Plain-text logs	`grep`, `awk`, `sed` with care	Fast local inspection and counting
Linux system journal	`journalctl`	Filter systemd journal by time, unit, priority and fields
Linux audit logs	`ausearch`	Search audit records by event type, key, user, time and success
Windows event logs	PowerShell `Get-WinEvent`	Filter channels, IDs, time windows and structured event properties

Do not mix query languages. A WQL expression is not automatically valid OpenSearch DSL, PowerShell or shell syntax.

Part I — Wazuh Query Language

2. WQL structure

WQL uses:

```
field operator value
```

Operators:

```
=   equal
!=  not equal
>   greater than
<   less than
~   like/matching text
```

Separators:

```
;   AND
,   OR
()  grouping
```

Examples:

```
status=active
status=active;os.name=Ubuntu
status=active,(os.name=Ubuntu,os.name=Debian)
name~web
```

Use double quotes around values containing spaces:

```
name="Windows Server 2025"
```

WQL is case-sensitive. Use the field suggestions and validation provided by the dashboard tab.

3. WQL cautions

- Field availability depends on the selected tab or API endpoint.
- `;` means AND and `,` means OR; this differs from many programming languages.
- The `~` operator is not a regular-expression guarantee. Confirm its behaviour in the specific endpoint.

- When using the API, URL-encode reserved characters or use a client option that performs encoding.
- WQL filters management/API data; indexed security-event searches may use another syntax.

4. WQL investigation record

```
Question: Which active agents report Ubuntu?
Interface: Agents management / supported WQL search
Query: status=active;os.name=Ubuntu
Time run: 2026-08-01 10:00 UTC
Result: [record count and summary]
Limitation: Agent status does not prove current event delivery for every configured source.
```

Part II — Dashboard field filters and Discover

5. Build searches from fields

In Discover or Threat Hunting:

1. select the correct data view, such as `wazuh-alerts-*`;
2. set an absolute time range;
3. inspect an event to confirm field names;
4. add include/exclude filters from field values;
5. display only useful columns;
6. sort by the relevant event timestamp;
7. record every inherited filter.

Useful VCC fields include:

```
pod_id
event_type
action
outcome
user
source_ip
session_id
correlation_id
event_id
schema_version
synthetic
```

Field paths can be nested or prefixed after Wazuh indexing. Inspect the actual event rather than assuming the path.

6. Exact versus analysed fields

Search engines may store a text field in two forms:

- analysed text, useful for word searching;
- keyword/exact value, useful for equality, grouping and aggregation.

If `user` searches return unexpected partial matches, inspect whether an exact/keyword form exists. Do not invent a `.keyword` suffix without checking the mapping.

7. Time-range discipline

Record:

- start and end in UTC;
- the dashboard time field;
- browser time-zone setting;
- whether the source also has `event_time` or `ingest_time`;
- any known delay or clock skew.

A relative range such as “last 15 minutes” is not reproducible later.

Part III — OpenSearch Query DSL

8. Basic exact query

Use Dev Tools only in an authorised local environment and only against approved indices.

```
GET wazuh-alerts-*/_search
{
  "size": 50,
  "query": {
    "term": {
      "data.pod_id": "pod-03"
    }
  },
  "sort": [
    { "timestamp": "asc" }
  ]
}
```

A `term` query expects an exact value and is normally appropriate for keyword-like fields. Confirm the mapping first.

9. Boolean filter

```
GET wazuh-alerts-*/_search
{
  "size": 100,
  "query": {
    "bool": {
      "filter": [
        { "term": { "data.pod_id": "pod-03" } },
        { "term": { "data.event_type": "authentication" } },
        { "term": { "data.user": "svc-backup" } },
        {
          "range": {
            "timestamp": {
              "gte": "2026-08-01T09:00:00Z",
              "lte": "2026-08-01T09:30:00Z"
            }
          }
        }
      ]
    }
  },
  "sort": [
    { "timestamp": "asc" }
  ]
}
```

Use `filter` for exact constraints that do not need relevance scoring.

10. Search one source across accounts

```
GET wazuh-alerts-*/_search
{
  "size": 100,
  "query": {
    "bool": {
      "filter": [
        { "term": { "data.pod_id": "pod-03" } },
        { "term": { "data.source_ip": "203.0.113.44" } },
        {
          "range": {
            "timestamp": {
              "gte": "2026-08-01T09:00:00Z",
              "lte": "2026-08-01T09:30:00Z"
            }
          }
        }
      ]
    }
  },
  "sort": [
    { "timestamp": "asc" }
  ]
}
```

11. Count by account

```
GET wazuh-alerts-*/_search
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        { "term": { "data.pod_id": "pod-03" } },
        { "term": { "data.event_type": "authentication" } },
        { "term": { "data.outcome": "failure" } },
        {
          "range": {
            "timestamp": {
              "gte": "2026-08-01T09:00:00Z",
              "lte": "2026-08-01T09:30:00Z"
            }
          }
        }
      ]
    }
  },
  "aggs": {
    "failures_by_user": {
      "terms": {
        "field": "data.user",
        "size": 20
      }
    }
  }
}
```

If the aggregation fails, verify that the field is aggregatable and mapped as an exact/keyword value.

12. Check field mappings

```
GET wazuh-alerts-*/_mapping/field/data.user
```

Use mappings to confirm type and exact-field behaviour. Do not change production mappings from a student lab.

13. Query DSL cautions

- `match` performs analysed full-text search and is not the same as `term`.
- Wildcards at the beginning of a value can be expensive.
- Large `size` values can overload the browser or indexer.
- Aggregation counts can be misleading when duplicate events exist.
- The `_source` document may contain sensitive fields; export only what the assignment requires.
- A successful query proves only that matching indexed documents were found.

Part IV — jq for NDJSON and JSON

14. Validate every NDJSON line

```
jq -c . sample-logs/authentication/failed-login-chain.ndjson >/dev/null
```

No output and exit code `0` means each non-empty line was valid JSON. It does not validate the event schema.

15. Display selected fields

```
jq -c '{event_time,event_id,event_type,user,source_ip,outcome,session_id}' \
  sample-logs/authentication/failed-login-chain.ndjson
```

16. Filter authentication failures

```
jq -c 'select(.event_type == "authentication" and .outcome == "failure")' \
  sample-logs/authentication/failed-login-chain.ndjson
```

17. Filter by account and sort by time

For NDJSON, first read the stream into an array:

```
jq -s '
  map(select(.user == "svc-backup"))
  | sort_by(.event_time)
  | .[]
  | {event_time,event_id,action,outcome,source_ip,session_id}
' sample-logs/authentication/failed-login-chain.ndjson
```

Use `-s` carefully with large files because it loads all records into memory.

18. Count failures by user

```
jq -s '
  map(select(.event_type == "authentication" and .outcome == "failure"))
  | group_by(.user)
  | map({user: .[0].user, count: length})
' sample-logs/authentication/failed-login-chain.ndjson
```

`group_by` expects sorted input by the grouping expression. jq sorts as part of `group_by`, but review output rather than assuming every missing user field behaves as intended.

19. Find duplicate event IDs

```
jq -r '.event_id' sample-logs/authentication/failed-login-chain.ndjson \
  | sort \
  | uniq -d
```

No output means no duplicate IDs were found in the examined file. It does not prove upstream collectors never duplicated semantically identical events under different IDs.

20. Check pod and synthetic markers

```
jq -s '{
  pods: (map(.pod_id) | unique),
  schemas: (map(.schema_version) | unique),
  all_synthetic: all(.[]; .synthetic == true)
}' sample-logs/authentication/failed-login-chain.ndjson
```

Part V — grep, awk and text logs

21. Fixed-string search

Prefer fixed-string mode when you do not need regular expressions:

```
grep -F 'Failed password' sample.log
```

22. Include line numbers and context

```
grep -n -C 2 -F '203.0.113.44' sample.log
```

23. Count matching lines

```
grep -c -F 'authentication failure' sample.log
```

Line count is not automatically event count. Multiline events and duplicate records can change the meaning.

24. Safe recursive search

```
grep -RIn --exclude='*.key' --exclude='.env' -F 'correlation-id' ./approved-log-folder
```

Run recursive searches only inside the authorised local working directory. Avoid accidentally scanning credential folders or unrelated personal files.

25. awk field example

For a controlled space-delimited sample:

```
awk '$9 == 401 {print $1, $4, $7, $9}' access.log
```

Real web log fields can contain quoted spaces and custom formats. `awk` column assumptions must be verified against the exact log format.

Part VI — journalctl

26. Query by unit and time

```
journalctl --unit=sshd.service \
  --since='2026-08-01 09:00:00 UTC' \
  --until='2026-08-01 10:00:00 UTC' \
  --no-pager
```

27. Query by priority

```
journalctl --priority=warning..alert --since='1 hour ago' --no-pager
```

28. Output JSON

```
journalctl --unit=sshd.service --since='today' --output=json --no-pager \
  | jq -c '{time:.__REALTIME_TIMESTAMP,unit:._SYSTEMD_UNIT,pid:._PID,message:.MESSAGE}'
```

`__REALTIME_TIMESTAMP` is normally expressed in microseconds since the Unix epoch. Convert carefully before mixing it with ISO 8601 timestamps.

29. Filter structured journal fields

```
journalctl _SYSTEMD_UNIT=ssh.service _COMM=sshd --since='today' --no-pager
```

Unit names differ between distributions. Confirm whether the system uses `ssh.service` or `sshd.service`.

30. journalctl cautions

- Access may require elevated local permissions.
- Volatile journals can disappear after reboot.

- Retention depends on journal configuration and storage.
- Service messages are application text inside a structured journal; parse them cautiously.
- A missing record may mean the unit did not log it, not that the action did not happen.

Part VII — ausearch

31. Failed login records

```
ausearch --message USER_LOGIN --success no --start today --interpret
```

32. Search by audit key

```
ausearch --key identity_changes --start today --interpret
```

The audit key exists only when an audit rule was configured with that key.

33. Unsuccessful system calls

```
ausearch --message SYSCALL --success no --start today --interpret
```

This can produce high volume. Narrow by time, user, executable or audit key where appropriate.

34. ausearch cautions

- Audit records for one action can span multiple related messages with the same audit event identifier.
- `--interpret` improves readability but may resolve numeric values using current system state.
- Audit rules must exist before the activity occurs.
- Use elevated permissions only on a system you own or are authorised to administer.

Part VIII — PowerShell and Windows Event Logs

35. List recent failed logons

```
$start = [datetime]'2026-08-01T09:00:00Z'
$end   = [datetime]'2026-08-01T10:00:00Z'

Get-WinEvent -FilterHashtable @{
    LogName   = 'Security'
    Id        = 4625
    StartTime = $start.ToLocalTime()
    EndTime   = $end.ToLocalTime()
} | Select-Object TimeCreated, Id, MachineName, RecordId
```

`Get-WinEvent` uses local `DateTime` values for the filter. Record how UTC times were converted.

36. Inspect structured event properties

```
$event = Get-WinEvent -FilterHashtable @{
    LogName = 'Security'
    Id      = 4625
} -MaxEvents 1

[xml]$xml = $event.ToXml()
$xml.Event.EventData.Data | ForEach-Object {
    [pscustomobject]@{
        Name  = $_.Name
        Value = $_.'#text'
    }
}
```

Property positions can differ between event versions. Named XML fields are safer than relying only on numeric property indexes.

37. Sysmon process creation

```
Get-WinEvent -FilterHashtable @{
    LogName   = 'Microsoft-Windows-Sysmon/Operational'
    Id        = 1
    StartTime = (Get-Date).AddHours(-1)
} | Select-Object TimeCreated, Id, RecordId, Message
```

The message is convenient for reading, but structured XML fields such as `ProcessGuid`, `Image`, `CommandLine`, `ParentProcessGuid` and hashes are stronger for correlation.

38. Export selected events

```
Get-WinEvent -FilterHashtable @{
    LogName = 'Microsoft-Windows-Sysmon/Operational'
    Id      = 1,3,22
} -MaxEvents 200 |
    Export-Csv -Path '.\approved-output\sysmon-review.csv' -NoTypeInformation
```

Before sharing, review exported command lines, user names, paths and addresses for sensitive information.

39. Windows event cautions

- Event 4624 proves a logon session was created, not that it was authorised.
- Event 4625 proves a logon attempt failed, not why the attempt occurred.
- Event 4688 command-line visibility depends on audit policy.
- Sysmon coverage depends on its installed version and configuration.
- Event messages can be localised; structured fields are more portable.
- Clearing or overwriting a log creates visibility limitations that must be documented.

Part IX — Query-writing method

40. Ten questions before drawing a conclusion

1. Which source produced the record?
2. Which time field am I using?
3. Which identity acted or was targeted?
4. Which asset, object or service was affected?
5. What action and outcome are recorded?
6. Which fields are original, decoded or enriched?
7. What preceding and following events matter?
8. What normal behaviour or maintenance context exists?
9. What does this result fail to prove?
10. Which next query would reduce uncertainty most?

41. Query journal standard

Every important query should record:

```
Question being tested:
Platform and source:
Exact query:
Time field:
Absolute UTC range:
Result count:
Important result:
Negative finding:
Evidence IDs:
Limitations:
Next pivot:
```

42. Common query mistakes

- using free text when an exact structured field exists;
- searching ingest time while interpreting the result as event time;
- leaving a hidden dashboard filter active;
- using `match` when exact equality is required;
- grouping duplicate events without checking IDs;
- treating zero results as proof before validating the source;
- pasting secrets or private URLs into commands, screenshots or reports;
- running commands outside the approved local or VCC scope;
- copying a query from another SIEM without translating field names and semantics.

Authoritative references

- Wazuh, *Filtering data using Wazuh Query Language*.
- Wazuh, *Wazuh indexer indices* and *Event logging*.
- OpenSearch, *Query DSL* and mapping documentation.
- jq manual.
- GNU grep and GNU awk manuals.
- systemd `journalctl` manual.
- Red Hat audit and `ausearch` documentation.
- Microsoft Learn, `Get-WinEvent`, Windows auditing and Sysmon documentation.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.